

Integrating symbolic execution with LLMs for automated generation of C program specifications

Fanpeng Yang, Xu Ma, Shuling Wang*, Xiong Xu, Qinxiang Cao, Naijun Zhan, Xiaofeng Li, and Bin Gu

Abstract—Automatically generating formal specifications including loop invariants, preconditions, and postconditions for legacy code is critical for program understanding, reuse, and verification. However, the inherent complexity of control and data structures in programs makes this task particularly challenging, especially when no verification goal is provided to drive synthesis. This paper presents SESPEC, a framework that integrates symbolic execution with large language models (LLMs) and a formal verifier (Frama-C/WP) to synthesize verifier-checked C program specifications. For loop-free fragments fully covered by the symbolic executor, SESPEC computes exact path-sensitive postconditions, composing the specifications of ordinary (non-recursive) callees along the way; at the remaining boundaries (loops, recursive data structures, and fragments outside the symbolic executor), it uses the available symbolic facts to guide LLM synthesis whose candidate specifications are then checked by the verifier. During symbolic execution, SESPEC decomposes the synthesis task by pausing at program boundaries, where the symbolic state is injected into annotation templates and LLM prompts. When a specification is faulty, verifier feedback is combined with symbolic information to refine the failed candidate rather than re-sampling blindly. This design supports goal-independent specification generation, because it derives functional relations from symbolic states and verifier feedback rather than relying solely on externally provided assertions. Through extensive evaluation across numerical, pointer, struct, and linked-list benchmark families and an additional goal-independent benchmark (SESPEC-400), SESPEC outperforms the state-of-the-art LLM-based tools across diverse program classes, including the goal-independent setting in which no verification target is provided.

Index Terms—Program specification generation, loop invariants, symbolic execution, large language models, formal verification.

I. INTRODUCTION

Across software domains, from general-purpose systems to long-lived industrial control codebases, a vast body of legacy code lacks formal specifications, i.e., precise, verifiable descriptions of program behavior such as preconditions, postconditions, and loop invariants. Automating specification generation is therefore essential for program comprehension,

reuse, and verification. The task is challenging due to the interplay of complex control structures (nested branches, loops, function calls) and data representations (structs, pointers, inductive datatypes). In this work, we tackle *goal-independent* functional specification generation: inferring verifier-checked preconditions, postconditions, and loop invariants that capture functional behavior without requiring a user-supplied assertion as the synthesis target. The goal is not to prove one preselected property, but to recover verifier-checked specifications that capture a program’s functional behavior, whether or not symbolic execution can derive them exactly.

Existing work on program specification generation has largely focused on loop invariants. Traditional symbolic methods such as constraint solving [?], [?], Craig interpolation [?], [?], [?], recurrence analysis [?], [?], shape analysis [?], and dynamic methods such as Daikon [?] are each restricted to particular invariant forms or program classes; recent ML-augmented or SMT-augmented methods [?], [?], [?] are similarly confined to numeric programs. LLM-based approaches have begun to broaden the program coverage, but tend to produce specifications that are either ill-formed, too weak, or both, and recent integrations with static/formal techniques [?], [?], [?], [?] only partially address this. AUTOSPEC [?] and Preguss [?] are goal-dependent (requiring a predefined verification target or RTE target) and degrade on complex data structures; SPECGEN [?] is goal-independent but only in the Java/JML setting, and “verifier-accepted” alone says nothing about strength. These limitations leave open the problem of generating goal-independent, semantically informative, verifier-checked specifications for C programs with loops, calls, pointers, structs, and inductive data.

To address this open problem, we propose SESPEC, a framework that integrates symbolic execution, LLM-based synthesis, and formal verification. The key idea is to let symbolic execution drive the process. The analysis proceeds through the program and pauses at program boundaries (loops, calls, and recursive constructs), turning one whole-function specification task into a sequence of smaller local synthesis problems. On loop-free segments fully handled by the flattened-memory symbolic executor, symbolic execution computes an exact path-sensitive postcondition, which is the strongest postcondition within that model; outside this envelope, symbolic execution provides partial semantic anchors for LLM synthesis. At each boundary, the symbolic state, i.e. the symbolic store of variable values, the path conditions, and the types of the in-scope variables, collectively the *local symbolic context*, is injected into invariant templates and LLM prompts. When a candidate fails, formal verification feedback

*Corresponding author: Shuling Wang (e-mail: wangsl@ios.ac.cn).

F. Yang, X. Ma, S. Wang, and X. Xu are with the Institute of Software, Chinese Academy of Sciences, and also with the University of Chinese Academy of Sciences, Beijing, China (e-mail: yangfp@ios.ac.cn; maxu@ios.ac.cn; wangsl@ios.ac.cn; xux@ios.ac.cn).

Q. Cao is with the School of Computer Science and School of Artificial Intelligence, Shanghai Jiao Tong University, Shanghai, China (e-mail: cao-qinxiang@sjtu.edu.cn).

N. Zhan is with the School of Computer Science, Peking University, Beijing, China (e-mail: znj@ios.ac.cn).

X. Li and B. Gu are with the Beijing Institute of Control Engineering, Beijing, China (e-mail: 1351127502@126.com; gubin@ios.ac.cn).

is combined with symbolic information to refine the failed candidate rather than re-sampling blindly.

We implement SESPEC on top of QCP’s symbolic execution engine [?], [?] and Frama-C/WP formal verifier [?]. We perform a comprehensive evaluation of SESPEC, including an ablation study that confirms the contribution of each component. We evaluate on benchmark families from prior invariant- and specification-generation work, covering numerical loops, multi-/nested loops, pointers, structs, and singly linked lists. On numerical benchmarks, SESPEC approaches the performance of tools specialized for numerical programs and consistently outperforms recent LLM-based approaches such as AUTOSPEC [?] and ACInv [?] (a static-analysis-guided LLM invariant generator). On the pointer-, struct-, and linked-list-heavy families that prior LLM-based tools handle poorly, it outperforms AUTOSPEC by a clear margin. We further construct **SESPEC-400**, a 400-program benchmark with all assertions removed, to stress-test the goal-independent setting at scale, and use it to analyze the validity, strength, failure modes, and cost of generated specifications; here SESPEC substantially outperforms the single-shot LLM and AUTOSPEC baselines, and the dimension-wise comparison finds that its specifications provide more informative postconditions and stronger loop invariants. Together these results show that the symbolic-execution-guided design generalizes from the goal-dependent to the goal-independent setting without specialization.

In summary, the key contributions of this work are:

- **Symbolic-execution-guided specification synthesis:** We present a framework that uses path-sensitive symbolic states to guide LLM-based specification generation. On segments fully covered by symbolic execution, these states provide exact semantic summaries; outside the fully covered fragment, the same symbolic state still supplies local symbolic context to anchor templates and prompts for LLM synthesis.
- **Verifier-guided iterative refinement:** We design a multi-layer refinement mechanism that classifies Frama-C/WP errors into diagnostic categories and applies targeted repair strategies, progressively guiding the LLM toward verifier-checked specifications instead of re-sampling blindly.
- **Goal-independent evaluation:** We construct **SESPEC-400**, a benchmark of 400 C programs with assertions removed, and use it to conduct what we believe is the first empirical study of goal-independent specification generation, evaluating validity, strength, failure modes, and per-program specification generation cost.

The rest of the paper is organized as follows. Section II introduces a motivating example and gives an overview of our approach. Section III and Section IV present the core techniques for pre-/postcondition specification generation and loop-invariant generation, respectively. Section V describes the verifier-guided refinement procedure, and Section VI presents the system design and implementation. Section VII reports the empirical evaluation under established benchmark protocols. Section VIII presents a goal-independent study on the **SESPEC-400** benchmark. Section IX gives representative case

studies. Section X discusses limitations and open challenges, and Section XI discusses threats to validity. Section XII surveys related work, and Section XIII concludes. Finally, the data-availability statement provides links to the datasets, implementation code, LLM configurations, and supplementary results.

II. A MOTIVATING EXAMPLE AND OUR APPROACH

Automating the generation of formal specifications for C code remains challenging due to the prevalence of complex language features such as loops, pointer manipulation, and data structures. Consider the code fragment shown in Fig. 1, which is adapted from a navigation component in an industrial control codebase. The code defines a struct `CheckCal` with three fields: an integer array `pkv[10]`, an integer field `len` representing the number of valid elements in the array, and an integer field `chksum` storing the sum of the array’s elements. Function `foo` then invokes `CheckCalFun` on a `CheckCal` pointer `pIp`, with a verification goal annotated by `/*@ assert ... */`. The callee function `CheckCalFun` takes a `CheckCal` pointer `pIp`, computes the sum of the `len` elements in the array `pkv` via a loop, and stores the result in the field `chksum` of `pIp`. This example exhibits diverse features: complex data structures (structs, arrays and pointers), a loop with inductive operations over arrays, and pointer-induced aliasing and side effects. Consequently, it faces the following challenges:

- **Loop reasoning:** The summation loop `for(; i<pIp->len; i++)` requires determining whether the initial state allows loop entry and, if the loop executes, establishing invariants strong enough to relate the partial sum to the processed prefix of the array.
- **Structs and Pointers:** The procedures access arrays and read and write struct fields through pointers, making it essential to account for aliasing and side effects in their specifications. For instance, the specification for function `CheckCalFun` must guarantee memory safety for dereferencing `pIp->pkv[i]`, precisely describe the write effect on `pIp->chksum`, and specify frame properties for fields of `pIp` that remain unchanged.
- **Inductive predicates:** Properties of arrays in loops, such as aggregate sums, are often most naturally expressed with inductive predicates or logic functions. Automatically writing such definitions in a verifier-friendly form and then proving the resulting annotations remains a central challenge for programs involving recursive operations or data structures.

As shown in Fig. 1, the code is annotated with the full ACSL specifications enclosed in `/* @... */`, including preconditions (beginning with `requires`), postconditions (beginning with `ensures`), loop invariants (beginning with `loop invariant`), and verification goals (beginning with `assert`). Among these, the loop invariants, the default preconditions, and the postconditions are generated automatically by our tool SESPEC, while the `assert` clause represents verification goals provided a priori, and the non-default preconditions shown on the designer-annotated `requires` lines



Fig. 1. Motivating Example

of CheckCalFun in Fig. 1 reflect additional requirements from the designers.

Before presenting our method, we attempted to generate the specification for this example using GPT-4o, Claude 3.7 Sonnet, and AUTOSPEC, given both the preconditions and the verification goal. The loop invariants they produced, shown below, were all found to be incorrect:

GPT-4o

```

loop invariant 0 <= i <= pIp->len;
loop invariant chksum ==
  \sum(0, i-1, \lambda integer k;
    pIp->pkv[k]);

```

CLAUDE 3.7 SONNET

```

loop invariant 0 <= i <= pIp->len;
loop invariant chksum ==
  \sum(integer k = 0; k < i; pIp->pkv[k]);
loop assigns i, chksum;
loop variant pIp->len - i;

```

AUTOSPEC

```

loop invariant 0 <= i <= pIp->len;
loop invariant chksum ==
  \sum(0, i-1, pIp->pkv);
loop invariant \forall integer k;
  0 <= k < i ==> chksum >= pIp->pkv[k];
loop assigns i, chksum;

```

The failures highlight common limitations of pure LLM-based approaches. First, all three state the partial-sum relation but none guards the case in which the loop never executes (e.g., $pIp \rightarrow len \leq 0$): they assume loop entry, so even the well-formed clauses fail the base case. Even when we explicitly prompted GPT-4o to “consider the case when the loop is not entered”, it still failed to produce the correct conditional invariant structure. Second, each tool invents a *different* syntax for the array aggregate— $\backslash\text{sum}$ with a lambda, with a quantified binder, or over a raw pointer—without defining it as an ACSL logic function; $\backslash\text{sum}$ is not a construct that Frama-C/WP discharges by default, so these clauses are rejected as undefined.

A. Our Approach

Fig. 2 shows an overview of our approach. Given a function and an optional verification goal, the objective is to automatically generate loop invariants and function pre-/postconditions. If a verification goal is provided, these specifications should be sufficient to prove it; otherwise, they should capture useful functional relations exposed by the available symbolic states. Following the three design choices of Section I, symbolic execution drives the pipeline and computes path-sensitive symbolic information at the current boundary; SESPEC injects these facts into invariant templates and ACSL skeletons, while a lightweight static route selects supporting examples and ACSL guidance. The LLM therefore receives both semantic facts from the current symbolic state and prompt support material matched to the program’s syntactic features. The verifier then checks the candidate, and if it fails, the parsed verifier feedback is combined with the same local symbolic context to drive iterative refinement until a verifier-checked summary is produced. Each generation call is issued at a symbolic-execution boundary — a loop, an unspecified callee, a recursive data structure, or a recursive function — and once a verifier-checked summary is obtained, symbolic execution resumes from the same point using that summary.

The **Function Specification Generation** process comprises four symbolic-execution-driven stages, as illustrated in Fig. 2, plus an **LLM-Fallback Layer** (Section III-D) that takes over whenever symbolic execution cannot produce a usable summary (self-recursive functions, recursive data structures, external calls, path explosion, floating-point arithmetic, or runtime errors). The four symbolic-execution-driven stages are described below. It begins with **Default Precondition Generation** (marked in green), which automatically produces memory-safety preconditions sufficient to start symbolic execution (node ① in Fig. 1); the procedure is detailed in Section III-B.

Using this precondition, the tool initiates symbolic execution from the function’s entry point (node ① in Fig. 1). When a function call is encountered (node ② in Fig. 1), the current symbolic execution is paused and **Callee Specification Generation** is invoked (marked in purple in Fig. 2). It recursively processes the callee function to generate its specification (nodes ③–⑥ in Fig. 1). Upon completion, the tool returns to the call site, incorporates the generated pre-/postcondition of the callee, and resumes symbolic execution of the caller. Throughout this process, a call stack is maintained to support nested function calls. Functions are processed in a bottom-up order, and all generated function specifications are stored for reuse across different calling contexts.

When symbolic execution encounters a loop within the current function (node ⑤ in Fig. 1), it pauses and transfers control to **Loop Invariant Generation** (marked in blue in Fig. 2). The Loop Analysis module is first invoked with the complete symbolic state at the loop entry and identifies different types of variables by analyzing the behavior of the loop body. In the presented example, the analysis produces the following results: the unchanged variables `pIp->len`, `pIp`, and `pIp->pkv`; and the inductive variables `i` and

`chksum`. The loop guard `i < pIp->len` is also recorded, together with the meta-condition that determines whether the loop body is executed at least once (here, `0 < \at(pIp, Pre)->len`). Subsequently, the tool generates parameterized invariant templates designed to capture various execution scenarios, such as whether the loop is entered, which variables remain unchanged, which variables evolve inductively, and how they behave. These templates are ACSL skeletons whose guards, unchanged-variable clauses, frame facts, and relation placeholders are derived from symbolic loop-entry and loop-body information. Below we present part of the generated templates for the motivation example (see Fig. 1 for the full set):

```
loop invariant (0 < \at(pIp, Pre)->len)
  ==> PLACE_HOLDER_for_i;
loop invariant (0 < \at(pIp, Pre)->len)
  ==> PLACE_HOLDER_for_chksum;
loop invariant pIp == \at(pIp, Pre);
```

These skeletons provide specific `PLACE_HOLDER` tags for the LLM to populate (marked as the blue boxes in Fig. 1). Guided by these structured templates and symbolic execution results, the LLM produces loop invariants that capture useful relations among different types of variables. This symbolic-state-guided structure narrows the candidate space and makes key logical constraints explicit before the LLM fills the placeholders. For instance, the generated invariants correctly incorporate the loop-entry condition (`0 < \at(pIp, Pre)->len`)—the critical logical constraint whose omission caused the pure-LLM failures in Section II.

During Loop Invariant Generation, the verifier-guided refinement procedure is iteratively invoked to assess both the validity of the generated invariants and their usefulness for proving the given verification goal when such a goal is available. Depending on verifier feedback, different refinement strategies guide the LLM toward verifier-checked invariants.

When symbolic execution of the callee is completed (node ⑥ in Fig. 1), **Function Postcondition Generation** (marked in red in Fig. 2) is triggered. The resulting symbolic states are used to synthesize a postcondition that summarizes the callee’s functional behavior. Upon completion, the generated pre-/postconditions are returned to the call site and reused as a functional summary in the caller (node ⑦ in Fig. 1), enabling symbolic execution of the caller to resume without re-analyzing the callee’s internal control flow.

This example illustrates the full pipeline; goal-independent generation is evaluated separately in Section VIII.

III. PRE-/POSTCONDITION GENERATION

This section first introduces symbolic-execution-based specification generation for code segments without loops, and then presents the generation of general function specifications. Loop-invariant generation is deferred to the next section.

A. Symbolic execution and strongest postcondition

Before stating the proposition, we fix its assumptions.

We use a *flattened memory model* to explicitly track memory values and aliasing during symbolic execution. At each

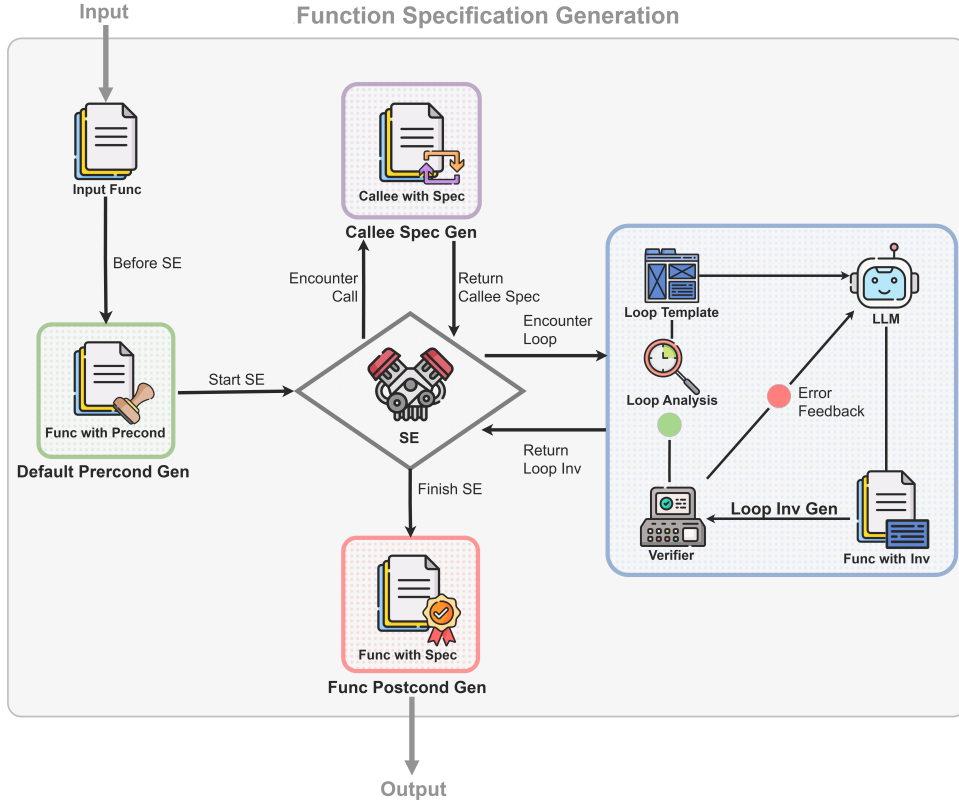


Fig. 2. Overview of Our Approach

program point p , the *scope* is the set of variables and formal parameters accessible at p . Starting from these roots, the model enumerates the finite set of *atomic memory locations* $\mathcal{L}_p$ reachable by dereferencing pointers and selecting struct fields and array elements: a struct is flattened field by field, an array of statically known or parameter-bounded length element by element, and each resulting primitive cell (a scalar or a pointer value), is represented by a distinct typed symbolic variable. Aliasing is expressed over these location names: access paths that the model resolves to the same cell denote the same symbolic location, while pairs that can be resolved as distinct are recorded as separated, thus equality, disjointness, and per-cell update facts are all stated over $\mathcal{L}_p$.

The model produces only finite, first-order facts over named cells; it does not use recursive or inductive predicates. These facts serve as preconditions and postconditions for two backends: QCP (in QCPL) for symbolic execution, and Frama-C/WP (in ACSL) for verification. What limits the expressiveness of the facts is Frama-C/WP's requirement for automatic SMT discharge: because the verification step must be fully automatic, the facts are confined to first-order and free of recursive/inductive predicates. Hence the model is deliberately finite. It handles scalars, structs, fixed arrays, and pointers with statically resolvable aliasing, but not unbounded or recursive structures (e.g., linked lists, trees), dynamic unbounded allocation, or accesses with a symbolic offset (e.g., pointer arithmetic). These cases fall outside the exact fragment and trigger the LLM fallback described in Section III-D.

Within this model, the symbolic execution of statements

maintains a *symbolic state* (PC_π, σ_π) for each execution path π . At a program point p on path π , let $\mathcal{L}_p^\pi$ denote the set of symbolic locations currently tracked by the executor. This set includes the entry-point locations $\mathcal{L}_{p_0}$ reachable from the function parameters, together with the local variables that have entered scope along π before reaching p (hence $\mathcal{L}_{p_0} \subseteq \mathcal{L}_p^\pi$).

- PC_π denotes the path condition, i.e., a logical formula over initial symbolic values and constants that encodes the conditions needed to follow the path π .
- σ_π denotes the symbolic store along path π , i.e., a mapping from each currently tracked symbolic location in $\mathcal{L}_p^\pi$ to a symbolic expression over the initial values.

The symbolic execution of a program is defined as a transformation over sets of symbolic states, branching at conditional control flow points. Formally, execution begins at the function entry point p_0 with the initial symbolic state (PC_0, σ_0) , where σ_0 assigns an initial symbolic value to each location in the entry point location set $\mathcal{L}_{p_0}$, and PC_0 is the precondition constraining those initial values. For a program point p , let the set of all symbolic states reachable from p_0 be $\{(PC_{\pi_i}, \sigma_{p,\pi_i})\}_{i=1}^m$, where π_i ranges over all paths from p_0 to p . The store $\sigma_{p,\pi}$ is a mapping, not a formula. We turn it into a formula over the tracked locations by equating each value at p with its symbolic expression:

$$\rho(\sigma_{p,\pi}) \equiv \bigwedge_{\ell \in \text{dom}(\sigma_{p,\pi})} \ell^p = \sigma_{p,\pi}(\ell).$$

Here ℓ^p is the value of ℓ at p , while the expression $\sigma_{p,\pi}(\ell)$ uses initial symbolic values. An unchanged tracked cell retains

its value at the starting point; access paths known to alias refer to the same cell. Locations outside the finite tracked set are outside the exact claim. The path condition PC_π also uses initial symbolic values and includes the starting precondition. Thus the reachable state pairs at p , projected to the tracked locations, satisfy $\bigvee_{i=1}^m (PC_{\pi_i} \wedge \rho(\sigma_{p,\pi_i}))$. At entry, the store relation is the identity relation, so the formula is PC_0 together with those identity equalities.

We implement a symbolic execution procedure $SEStart(\mathcal{F}, P, p_{start})$. Given a code segment $\mathcal{F}$ and a precondition P (a formula representing symbolic states) at a starting point p_{start} , the procedure executes symbolically from p_{start} , exploring the feasible paths until it reaches the next program boundary (i.e. a loop entry, a function call, or the exit of $\mathcal{F}$), and returns the point p at which it pauses. By construction, the segment from p_{start} to p contains no loop and no function call, so its input/output behavior can be summarized exactly.

For an interval from p_{start} to p_{end} , “initial” means the state at p_{start} , even when it is not the function entry. In the proposition, $\mathcal{L}_{p_{end}}$ denotes the same observable endpoint locations tracked on every feasible path; path-local locations are omitted. The claim concerns initial values and these common endpoint locations. We first define the notion of a *strongest postcondition* in the Hoare-logic sense before stating the proposition for loop-free segments.

Definition 1. (Strongest Postcondition) For a segment $\mathcal{F}$ and a precondition P , a formula Φ over the tracked initial and reached state variables is the strongest postcondition of $\mathcal{F}$ with respect to P , if

- $\{P\} \mathcal{F} \{\Phi\}$ holds, meaning every state pair (s_0, s) reachable by executing $\mathcal{F}$ from $s_0 \models P$ satisfies Φ .
- For any Ψ over the same tracked state-pair vocabulary such that $\{P\} \mathcal{F} \{\Psi\}$ holds, we have $\Phi \Rightarrow \Psi$ as a relation over state pairs.

Let $\mathcal{F}$ be a program segment from p_{start} to p_{end} and P be a logical formula denoting the precondition. Write $\Pi = Path(p_{start}, p_{end})$ for the feasible paths explored under P , and $\Phi_{SP} = SP(\mathcal{F}, P, p_{start}, p_{end})$. We define

$$\Phi_{SP} = \bigvee_{\pi \in \Pi} (PC_\pi \wedge \rho(\sigma_{p_{end}, \pi} |_{\mathcal{L}_{p_{end}}}))$$

Then we state the following proposition.

Proposition 1. $SP(\mathcal{F}, P, p_{start}, p_{end})$ is the strongest postcondition of $\mathcal{F}$ with respect to P , if the following conditions hold:

- (A1) *Scope conditions:* $\mathcal{F}$ contains no external calls (i.e., every called function has a definition), no loops, and every memory access is representable in the flattened model. For every input satisfying P , execution of the interval is well-defined under the modeled C semantics: memory accesses, aliasing, and arithmetic operations are represented exactly and no undefined behavior occurs.
- (A2) *Symbolic execution conditions:* The symbolic execution procedure is path-complete, that is, it enumerates all feasible paths from p_{start} to p_{end} under P . Furthermore, for each enumerated path π , the path condition PC_π

exactly characterizes the inputs that cause the concrete execution to follow π , and the symbolic store $\sigma_{p_{end}, \pi}$ exactly expresses the resulting concrete state at p_{end} as a function of the initial state, according to the flattened memory semantics. These statements concern the common endpoint locations; other endpoint locations are not characterized by the formula.

Proof. Denote $SP(\mathcal{F}, P, p_{start}, p_{end})$ by Φ . We show that its satisfying state pairs are exactly the reachable pairs projected to the common endpoint locations; this establishes both soundness and maximality over those variables.

(Soundness): Take any initial state s_0 satisfying P . Since $\mathcal{F}$ is loop-free and contains no external calls, and (A1) excludes undefined behavior on this input, its concrete execution follows a feasible path π and reaches p_{end} in state s . By (A2) path-completeness, symbolic execution enumerates π . By (A1) and (A2), its flattened-memory transfers agree with the concrete execution on every tracked cell. Consequently, PC_π holds and $\ell^{p_{end}} = \sigma_{p_{end}, \pi}(\ell)$ for every $\ell \in \mathcal{L}_{p_{end}}$ under the pair (s_0, s) . Hence this pair satisfies $PC_\pi \wedge \rho(\sigma_{p_{end}, \pi} |_{\mathcal{L}_{p_{end}}})$ and therefore Φ . This proves that every reachable pair satisfies Φ .

(Completeness): Let (s_0, s) satisfy Φ . Some feasible path π then satisfies PC_π and $\rho(\sigma_{p_{end}, \pi} |_{\mathcal{L}_{p_{end}}})$ under this pair. Since PC_π includes P , the initial state satisfies P . Exact path conditions and transfers in (A2), together with (A1), give a defined concrete execution along π whose final state agrees with s on the common endpoint locations. Thus the projected pair is reachable. If $\{P\} \mathcal{F} \{\Psi\}$ holds for any formula Ψ over the same projected state variables, every reachable projected pair satisfies Ψ , so $\Phi \Rightarrow \Psi$. This proves the strongest-postcondition claim. $\square$

If symbolic execution is unexpectedly interrupted by unsupported program features, SP returns *SymExecFailed* and the affected task is handled by the LLM-based fallback layer (Section III-D). If symbolic execution encounters undefined behavior, it reports an error instead of returning an exact strongest postcondition. The fallback may still produce a verifier-checked specification, but it no longer carries the strongest-postcondition guarantee of Proposition 1.

B. Default Precondition Generation

The default precondition generator $DefaultPre(\mathcal{F})$ generates sufficient preconditions for the supported symbolic-execution model. By default, it assumes that differently named pointers refer to disjoint memory regions and imposes configured bounds on array lengths and values. These are default generation choices, not necessary conditions for all well-defined C executions. The separation assumption does not imply that SESPEC cannot handle aliasing; it restricts the input domain of contracts containing the corresponding separation clauses.

The generator processes the function parameters: for each pointer (including those inside structs and arrays), it emits a validity fact; after all pointers are collected, it appends a single separation fact asserting that the initial memory regions of these pointers are pairwise disjoint. Bound constraints

restrict array lengths and integer values to the configured analysis ranges (e.g. $0 < arr_len < MAX_LEN$, $0 \leq v \leq MAX_VAL$ for integer value v). This model is chosen because it lies at the intersection of the expressive power of QCP (our symbolic execution engine) and Frama-C/WP (our back-end verifier). The default generator does not emit additional functional assumptions such as sortedness.

Designers may supply additional precondition clauses; these are conjoined with the auto-generated precondition into PC_0 at p_0 before symbolic execution begins. Fig. 1 illustrates both modes: the validity and separation facts on `CheckCalFun` are auto-generated, while the range constraints on its array values and length field are designer supplied.

In contrast, for recursive pointer-linked data structures (linked lists, trees, recursive structs), the number of reachable memory cells is determined only at runtime (by the list length or tree depth). Therefore, the static generator cannot emit a finite set of validity/separation facts for the entire shape; instead, the shape must be captured by an inductive predicate. In this case `DefaultPre` delegates the generation of the inductive shape predicate to the LLM fallback layer (Section III-D), which proposes the predicate and the corresponding precondition. The non-recursive part (validity of the head pointer and of primitive fields) is still emitted by the static generator. The LLM-generated predicate and precondition clauses are kept only if Frama-C/WP accepts them.

C. Function Specification Generation Algorithm

Algorithm 1 describes the process of generating function specifications guided by symbolic execution. The algorithm takes as inputs a function $\mathcal{F}$, a stack cS of callee functions whose specifications are not yet available, and a list aF of functions already annotated with generated specifications. It performs symbolic execution throughout the procedure, recursively invoking callee-specification generation and loop-invariant generation as needed. Upon termination, it returns an annotated version of $\mathcal{F}$ equipped with pre-/postconditions and loop invariants.

The algorithm works as follows. If $\mathcal{F}$ has already been annotated, it returns immediately (Lines 1–2); otherwise $\mathcal{F}$ is pushed onto the call stack cS — which records functions encountered during execution that still lack specifications — and a fresh loop stack lS is initialized for the nested loops of $\mathcal{F}$ (Lines 3–4). The default memory-safety precondition P is then computed by `DefaultPre` (Section III-B) at the entry point p_0 , which assigns initial symbolic values to all atomic memory locations reachable from the variables in scope at p_0 (typically the function parameters), and symbolic execution begins from p_0 via `SEStart` (Lines 5–7); `SEStart` returns the next pause point p — the next loop entry, callee site, or function exit. While p has not reached the exit point p_{-1} (Line 8), the exact postcondition holding at p is aggregated through SP over the covered loop-free interval from the symbolic-execution results and p_{start} is reset to p (Lines 9–10), after which the dispatch is by boundary kind (Lines 11–15): a loop entry triggers `LoopInvGen`, invoked with P as the loop precondition and p as the loop entry, returning $\mathcal{F}$

Algorithm 1: Function Specification Generation

FuncSpec

Input: function $\mathcal{F}$, call stack cS , annotated functions aF
Output: function $\mathcal{F}$ with specification S

```

1 if  $\mathcal{F} \in aF$  then
2   return
3  $cS.push(\mathcal{F})$ ;
4  $lS \leftarrow \text{InitLoopStack}()$ ;
5  $P \leftarrow \text{DefaultPre}(\mathcal{F})$ ;
6  $p_{start} \leftarrow p_0$ ;
7  $p \leftarrow \text{SEStart}(\mathcal{F}, P, p_{start})$ ;
8 while  $p \neq p_{-1}$  do
9    $P \leftarrow SP(\mathcal{F}, P, p_{start}, p)$ ;
10   $p_{start} \leftarrow p$ ;
11  if  $p = p_{loop}$  then
12     $\mathcal{F} \leftarrow \text{LoopInvGen}(\mathcal{F}, p, lS, cS, aF, P, LLM)$ ;
13  if  $p = p_{callee}$  then
14     $\mathcal{F}_{callee} \leftarrow \text{FuncSpec}(\mathcal{F}_{callee}, cS, aF)$ ;
15   $p \leftarrow \text{SEStart}(\mathcal{F}, P, p_{start})$ ;
16  $S \leftarrow SP(\mathcal{F}, P, p_{start}, p_{-1})$ ;
17  $\mathcal{F} \leftarrow (\mathcal{F}, S)$ ;
18  $aF.append(cS.pop())$ ;
19 return  $\mathcal{F}$ ;

```

with the loop annotated by its invariant; a callee site triggers a recursive `FuncSpec` on the callee to obtain its specification; and in either case symbolic execution then resumes from the updated p_{start} under the freshly produced annotations. The two stacks together implement a compositional, bottom-up traversal: callee specifications are derived first and reused across calling contexts, and nested loops are processed before their enclosing loops. When symbolic execution reaches the function exit, SP aggregates the final postcondition S , the annotated $\mathcal{F}$ is recorded in aF , and $\mathcal{F}$ is popped from cS and returned (Lines 16–19).

Two design points are worth making explicit. First, Algorithm 1 presents the idealized symbolic-execution-driven main path. In practice, the LLM-fallback layer described in Section III-D wraps this algorithm rather than being inlined into it. The fallback handles cases where symbolic execution cannot produce a usable summary (self-recursive functions, recursive data structures, external calls, *etc.*). Second, the loop-handling counterpart `LoopInvGen` (Algorithm 2), invoked at Line 12, is also guided by symbolic execution. It instantiates invariant templates using the loop precondition P , the symbolic store and path conditions at the loop entry, and the unchanged and non-inductive variable sets extracted by `LoopAnalysis` (Section IV-C). The LLM then completes the template-specific invariant clauses, and the verifier-guided `Refine` step repairs them against Frama-C/WP feedback.

D. LLM Fallback

Algorithm 1 presents the idealized symbolic-execution-driven path and intentionally omits the fallback layer; the LLM-fallback layer wraps that algorithm and is triggered in exactly two cases. (i) When the target function is self-recursive or manipulates recursive data structures: the symbolic executor neither computes recursive fixpoints nor enumerates the un-

bounded cell set of a recursive shape, so the fallback is invoked directly at the function entry instead of running the algorithm. (ii) When $SP(\cdot)$ at the function exit returns *SymExecFailed*: this covers the residual cases in which symbolic execution cannot produce a usable summary, including external calls, path explosion, and runtime errors. Loops and ordinary callees do not enter fallback; they are handled by LoopInvGen and recursive FuncSpec respectively. Their residual verifier failures are sent to the verifier-guided refinement layer, which also wraps Algorithm 1 rather than being inlined.

In the fallback, the LLM acts as a semantic summarization component for the missing symbolic-execution step. It receives the current code fragment, available symbolic states and path conditions, previously generated callee specifications or loop summaries, and an ACSL skeleton. The LLM is tasked with following the relevant paths, tracking changes to variables and memory locations, and proposing ACSL clauses that summarize the observed behavior. The resulting candidates must still be proved by Frama-C/WP.

We use this fallback for four function-level tasks. *Precondition generation* asks the LLM to infer candidate *requires* clauses describing the validity, separation and bound constraints. *Assigns-clause generation* asks the LLM to infer the function's footprint, i.e., *assigns* clauses describing which visible memory locations may be modified. *Postcondition generation* asks the LLM to summarize the relation between the pre-state and the post-state, filling candidate *ensures* clauses. *Inductive-predicate generation* asks the LLM to decide whether the program requires an inductive predicate and, when needed, to propose one for capturing inductive relations in recursive data structures or recursive functions.

Fallback candidates are accepted only when Frama-C/WP proves them under the configured memory model and solver settings; verifier failures are handled by the refinement loop in Section V (Table I). Fallback can therefore cover cases beyond the exact symbolic-execution fragment but unlike the postconditions computed by SP (Proposition 1), the results produced by the fallback are sound over-approximations verified by the prover.

IV. LOOP INVARIANT GENERATION

The general loop invariant generation problem can be defined as follows:

Definition 2. Given a loop $l: \text{while } B \text{ do } C$ of program $\mathcal{P}$, a precondition P that holds at entry l , and an (optional) postcondition Q , the loop invariant generation problem is to find a logical formula Inv that satisfies the following conditions:

- **Base:** $P \Rightarrow Inv$, i.e., the invariant Inv holds initially;
- **Preservation:** $\{Inv \wedge B\} C \{Inv\}$, i.e., if Inv holds at the start of a loop iteration and loop condition B is true, executing loop body C preserves Inv ;
- **Termination:** $Inv \wedge \neg B \Rightarrow Q$, i.e., upon loop termination, B turns false, Inv remains true and implies postcondition Q .

Among these conditions, the postcondition serves as the verification target, whose objective is to synthesize sufficiently

strong invariants that imply it. Our invariant generation algorithm, presented below, is designed to produce strong invariants even in the absence of postconditions. We achieve this through a combination of symbolic execution and Large Language Models (LLMs), using symbolic-state-guided invariant templates that expose path conditions, variable classifications, frame facts, and relation placeholders derived from symbolic loop-entry and loop-body information. The effectiveness of this approach is experimentally demonstrated in Section VII-E.

Algorithm 2: Loop Invariant Generation LoopInvGen

Input: function $\mathcal{F}$, loop entry p_{loop} , loop stack ls , call stack cS , annotated functions aF , loop precondition P , LLM

Output: function $\mathcal{F}$ annotated with loop invariant $\mathcal{I}$ for the loop $\mathcal{F}_{loop}$ at p_{loop}

// $\mathcal{F}_{loop}$ is the loop body at p_{loop} in $\mathcal{F}$

- 1 ThinkInNaturalLanguage($\mathcal{F}_{loop}$, LLM);
- 2 $ls.push(\mathcal{F}_{loop})$;
- 3 $\mathcal{F}_{loop}, uV, nV \leftarrow \text{LoopAnalysis}(\mathcal{F}_{loop}, P, ls, cS, aF, LLM)$;
- 4 $ls.pop()$; // ls depth is unchanged across this call
- 5 $\mathcal{T} \leftarrow \text{LoopInvTemGen}(\mathcal{F}_{loop}, P, uV, nV)$;
- 6 $\mathcal{I} \leftarrow \text{OuterLoopLLMCall}(\mathcal{F}, \mathcal{F}_{loop}, P, \mathcal{T}, LLM)$;
- 7 $\mathcal{F} \leftarrow (\mathcal{F}, \mathcal{I})$;
- 8 **if** $\neg \text{Verify}(\mathcal{F})$ **then**
- 9 $\mathcal{F} \leftarrow \text{Refine}(\mathcal{F}, \mathcal{I}, P, loop, LLM)$;
- 10 **return** $\mathcal{F}$;

A. Invariant Generation for Loops

Algorithm 2 presents the invariant-generation procedure for a loop in $\mathcal{F}$. Among the inputs, p_{loop} denotes the loop entry in $\mathcal{F}$, P is the loop precondition obtained from previous symbolic execution, and the LLM serves as the synthesis component. Other parameters remain as previously defined.

The procedure separates three concerns. First (Line 1), the original outermost loop body $\mathcal{F}_{loop}$ is sent to the LLM for a chain-of-thought reading in natural language, in which the LLM is asked a few guiding questions to build a comprehensive understanding of the loop; this reading is used only as semantic support for later template filling and is never emitted as an annotation. Second (Lines 2–4), $\mathcal{F}_{loop}$ is pushed onto the loop stack and analyzed by *LoopAnalysis*, which symbolically executes the loop body, recursively handles all inner loops and annotates them with their invariants, and returns the sets of unchanged variables uV and non-inductive variables nV ; the outermost loop is then popped, so the stack depth is unchanged across the call. Third (Lines 5–9), the invariant for the outermost loop is synthesized: the results of *LoopAnalysis* feed *LoopInvTemGen* (Line 5), which emits the template family for the current loop; the LLM then completes these templates through a single *OuterLoopLLMCall* (Line 6); and if the resulting annotation fails verification, a single call to the unified verifier-guided refinement procedure *Refine* (Section V, Lines 8–9) refines the candidate until it passes verification.

Algorithm 3: Loop Analysis

Input: loop body $\mathcal{F}_{loop}$, precondition Pre , loop stack IS , call stack cS , annotated functions aF , LLM

Output: annotated $\mathcal{F}_{loop}$, sets of unchanged variables uV and non-inductive variables nV

```

1  $\mathcal{F}_{unfolded} \leftarrow \text{UnfoldLoop}(\mathcal{F}_{loop});$ 
2  $P \leftarrow Pre;$ 
3  $p_{start} \leftarrow p_{unfolded\_0};$ 
4  $p \leftarrow \text{SEStart}(\mathcal{F}_{unfolded}, P, p_{start});$ 
5 while  $p \neq p_{unfolded\_n-1}$  do
6    $P \leftarrow SP(\mathcal{F}_{unfolded}, P, p_{start}, p);$  // Same as FuncSpec
7    $p_{start} \leftarrow p;$ 
8   if  $p = p_{nestedLoopEntry}$  then
9      $IS.push(\mathcal{F}_{nested});$ 
10  if  $p = p_{nestedLoopExit}$  then
11     $\mathcal{F}_{nested} \leftarrow IS.pop();$ 
12     $\mathcal{F}_{loop} \leftarrow \text{NestedLoopLLMCall}(\mathcal{F}_{loop}, \mathcal{F}_{nested}, LLM);$ 
13  if  $p = p_{callee}$  then
14     $\mathcal{F}_{callee} \leftarrow \text{FuncSpec}(\mathcal{F}_{callee}, cS, aF);$ 
15   $p \leftarrow \text{SEStart}(\mathcal{F}_{unfolded}, P, p_{start});$ 
16 ...; // Same as FuncSpec, construct the final postcondition  $S$ 
17  $uV \leftarrow \text{FetchUnchangedVars}(S);$ 
18  $nV \leftarrow \text{FetchNonInductiveVars}(S);$ 
19 return  $\mathcal{F}_{loop}, uV, nV;$ 

```

B. Loop Analysis

LoopAnalysis (Algorithm 3) symbolically executes a single-iteration unfolding $\mathcal{F}_{unfolded}$ of the loop body (Lines 1–4) and then advances through it one program point at a time (Lines 5–15), exposing the update and frame facts of one iteration. The key difference from FuncSpec lies in the handling of nested loops: whereas the outer loop uses symbolic-execution-derived preconditions and templates to guide LLM synthesis (OuterLoopLLMCall), an inner loop is processed purely through NestedLoopLLMCall (Lines 10–12), which infers its invariant directly from the code context without preconditions or templates, because a well-defined precondition is not available at the enclosing loop boundary. Nested loops are thus discharged bottom-up using the loop stack IS , while function calls are handled consistently with FuncSpec through the call stack cS (Lines 13–14): the callee is summarized before symbolic execution of the unfolded body resumes. Once the unfolded body has been fully executed, its final postcondition is constructed and all nested loops are annotated with the invariants generated for them; this postcondition is then used to classify the unchanged variables uV and non-inductive variables nV of the outer loop (Lines 16–18).

C. Template Generation

The LoopInvTemGen procedure takes as inputs the loop body, the loop precondition, and the sets of unchanged and non-inductive variables, and generates invariant templates. These templates have parameters partially determined by symbolic execution results, while the remaining undetermined parameters are completed using the subsequent LLM synthesis method (i.e., OuterLoopLLMCall in Algorithm 2). The

prompt support material used during instantiation, including selected examples and ACSL guidance, is constructed as described in Section VI-B. Below we introduce six forms of invariant templates. The first four are deterministically derived from symbolic execution and cover variables whose behavior is captured by SP ; the fifth is a goal-dependent template that is enabled only when a verification goal is provided at the loop boundary; the sixth is a deliberately free-form *sink* template that lets the LLM contribute invariants involving quantifiers and inductive predicates, which symbolic execution cannot anchor. Without this sink, loop invariants that rely on relations not enumerated by symbolic execution, such as quantified array properties or recursive-data-structure predicates, would have no skeleton to complete and could not be expressed.

- **Unchanged variables:** For any variable $v \in uV$ and any execution path π_j to the loop entry p , v retains its symbolic value $\sigma_{p,\pi_j}(v)$ at the loop entry:

$$PC_{\pi_j} \Rightarrow v = \sigma_{p,\pi_j}(v)$$

- **Loop skip conditional property:** For a path π_j to p and a loop condition B , if B for path π_j is false, the loop is skipped without altering variables:

$$\neg(B \wedge (PC_{\pi_j} \wedge \rho(\sigma_{p,\pi_j}))) \Rightarrow v = \sigma_{p,\pi_j}(v)$$

- **Non-inductive variables:** For a non-inductive variable v (whose value is updated independently of its value from the previous iteration) and any path π_j to p :

$$(B \wedge (PC_{\pi_j} \wedge \rho(\sigma_{p,\pi_j}))) \Rightarrow (v = \sigma_{p,\pi_j}(v)) \vee \Phi(v)$$

where $\Phi(v)$ is the inductive invariant to be synthesized.

- **Inductive variables:** For an inductive variable v (which is neither unchanged nor non-inductive) and any path π_j to p :

$$(B \wedge (PC_{\pi_j} \wedge \rho(\sigma_{p,\pi_j}))) \Rightarrow \Phi(v)$$

- **Verification goals:** In addition to variable-specific templates, we also introduce templates corresponding to each verification goal. For a verification goal g and any path π_j to p :

$$(B \wedge (PC_{\pi_j} \wedge \rho(\sigma_{p,\pi_j}))) \Rightarrow \Phi(g)$$

- **Free-form sink:** an unanchored slot Φ that lets the LLM propose invariants involving quantifiers and inductive predicates that the deterministic forms above cannot anchor.

This template-driven approach is crucial for our loop invariant generation algorithm. By leveraging information derived from symbolic execution, it simplifies the LLM's task, shifting from full invariant generation from scratch to completing invariant templates. It also offers structural guidance, helping the LLM produce formulas that conform to the required ACSL logical syntax. More importantly, this approach encourages strong invariants that capture relevant variable behaviors across feasible execution paths, while the final verifier-guided refinement step determines which generated clauses can be accepted by Frama-C/WP.

V. VERIFIER-GUIDED SPECIFICATION REFINEMENT

Refinement is invoked after SESPEC has produced candidate loop invariants or function specifications. A candidate can fail in three different ways: it may be syntactically incorrect (failing to conform to ACSL grammar), it may be semantically invalid (failing to be proved by Frama-C/WP), or logically weak (verifiable but too weak to prove a downstream assertion when such a target exists). The refinement procedure takes five inputs: the annotated program $\mathcal{F}$, the candidate annotation region $\mathcal{A}_0$ (the loop invariant or specification clauses to be repaired, while the rest of $\mathcal{F}$, denoted by $\mathcal{F}^-$, remains fixed), the local symbolic context Γ saved when the candidate was generated, the target kind $k \in \{\text{loop}, \text{spec}\}$, and the LLM. The output is $\mathcal{F}^*$ with a verifier-checked annotation region $\mathcal{A}^*$ when the iteration converges, or the strongest verifier-checked subset found after budget exhaustion.

Algorithm 4 shows the control flow. The algorithm first separates the candidate annotations from the surrounding program, storing the latter as $\mathcal{F}^-$, so every refinement round edits only the current annotation region. Each iteration runs Frama-C/WP on the current $\mathcal{F}$. If verification succeeds, the loop stops. If the verifier reports a syntax or type error before semantic proof begins, refinement follows the syntax-repair branch. Otherwise, the proof failures are classified by their proof role and the target kind k ; only the non-empty failure categories are rendered into the next prompt. If the iteration budget is exhausted, the algorithm switches to a conservative Houdini-style pruning pass.

Algorithm 4: Verifier-Guided Refinement Refine

Input: $\mathcal{F}$ with candidate annotation $\mathcal{A}_0$, symbolic context Γ , target kind $k \in \{\text{loop}, \text{spec}\}$, LLM
Output: $\mathcal{F}^*$ with verifier-checked annotation $\mathcal{A}^*$

```

1  $\mathcal{A} \leftarrow \mathcal{A}_0$ ;
2  $\mathcal{F}^- \leftarrow \mathcal{F} - \mathcal{A}_0$ ;
3 while  $Errors \leftarrow \neg \text{Verify}(\mathcal{F})$  do
4   if reach iteration limit then
5     // Layer 3: Houdini-style pruning
6      $\mathcal{A} \leftarrow \text{Houdini}(\mathcal{F}^-, \mathcal{A}, Errors)$ ;
7      $\mathcal{F} \leftarrow (\mathcal{F}^-, \mathcal{A})$ ;
8     break;
9   if  $Errors = \text{syntax error}$  then
10    // Layer 1: syntax repair
11     $\mathcal{A} \leftarrow \text{RepairCallLLM}(\mathcal{A}, \Gamma, Errors)$ ;
12  else
13    // Layer 2: strategy-driven
14    // refinement or regeneration
15     $guidance \leftarrow \text{BuildGuide}(\text{Classify}(Errors, k), \Gamma)$ ;
16     $\mathcal{A} \leftarrow \text{RefineCallLLM}(\mathcal{A}, \Gamma, guidance)$ ;
17   $\mathcal{F} \leftarrow (\mathcal{F}^-, \mathcal{A})$ ;
18 return  $\mathcal{F}$ ;
```

The verifier-guided refinement stage is implemented as three layers, matching the three branches in Algorithm 4.

Layer 1: syntax repair. Layer 1 handles syntax and type errors. If Frama-C rejects the candidate before generating proof obligations, SESPEC calls the LLM with only the current annotation region and the corresponding verifier error. The returned annotation is checked again by Frama-C/WP. This

layer only makes the candidate parsable; semantic repair is handled by Layer 2.

Layer 2: strategy-driven semantic refinement or regeneration. When the candidate parses but Frama-C/WP fails to prove it, SESPEC classifies the failures into typed categories such as loop-invariant obligations, function-specification obligations, callee-instance obligations, frame obligations, and downstream assertion obligations. The complete category-to-strategy mapping is given in Table I. For each round, SESPEC loads only the strategy blocks whose categories are non-empty, so the prompt exposes the formal failure together with its matching fix strategy. This is the main refinement step: it revises invalid clauses, strengthens valid but insufficient clauses, or regenerates an unstable annotation region when the current candidate is globally inconsistent.

Layer 3: Houdini pruning. When the refinement budget is exhausted, SESPEC runs one Houdini-style deletion pass over the current annotation region. For loop refinement, it removes still-failing loop invariants while preserving the largest verifier-checked subset. For function-level refinement, it removes failing target `ensures` clauses and residual loop invariants. This layer is intentionally conservative: it only deletes clauses identified by verifier feedback and returns the strongest subset accepted by the verifier.

The same algorithm serves loop-level and function-level refinement; only the set of error categories produced by $\text{Classify}(Errors, k)$, and hence the strategy rows selected from Table I differs. This keeps the control loop uniform while letting each failure category receive its own repair action. *Implementation note (best-candidate rollback).* Algorithm 4 presents the logical refinement skeleton. In the implementation, SESPEC additionally tracks the best candidate seen so far, using a lexicographic score

$$\text{Score}(\mathcal{F}) = (\mathbf{1}_{\text{parse}}, \frac{|\text{Valid}(VC)|}{|VC|}, |C|),$$

where VC is the set of verification conditions generated by Frama-C/WP for the candidate, $\text{Valid}(\cdot)$ is the subset proved, C is the number of retained clauses, and $\mathbf{1}_{\text{parse}}$ is 1 iff the candidate parses. Candidates are ordered lexicographically by this score, preferring parseability, then proof ratio, then the number of retained clauses. If two consecutive rounds do not improve the best score (*patience* = 2), the implementation restores the best recorded candidate and resumes refinement from that snapshot. If the iteration budget is exhausted, Houdini starts from the same best candidate.

VI. SYSTEM DESIGN AND PROMPT CONSTRUCTION

A. Execution and Verification Backbone

The implementation combines three off-the-shelf components: the QCP symbolic execution engine [?], [?], an LLM, and the Frama-C/WP automated verifier [?], and orchestrates them through a thin translation layer. Symbolic execution drives the pipeline and pauses at each loop and function call, where the LLM proposes the missing annotations from the exposed symbolic state and Frama-C/WP discharges the resulting proof obligations.

TABLE I
REFINEMENT STRATEGIES BY VERIFIER-FAILURE CATEGORY.

Target	Verifier feedback	Refinement strategy
Loop invariant	Establishment (base) obligation fails	Drop or guard facts not implied by the pre-loop state.
	Preservation obligation fails	Rebuild the inductive relation from the loop body's one-step effect.
	Invariant holds but the downstream goal or assertion is unproved	Add loop-exit consequences linking the negated guard to the inductive and unchanged facts.
	Several invariants fail or the candidate is globally inconsistent	Discard the candidate and regenerate from the original template.
Function specification	Target postcondition obligation fails	Drop over-strong claims or split them into path-sensitive clauses.
	Callee-instance obligation fails at a call site	Strengthen caller-side facts before the call, or weaken an over-strong callee precondition.
	Postcondition holds but a user assertion still fails	Add the missing bridge facts to the specification or to the loop-exit facts.
	Frame (assigns) obligation fails	Add only the missing externally visible locations.

The choice of QCP and Frama-C is motivated by their complementary strengths. QCP is used as the symbolic execution engine because it produces Coq-verified postconditions and, more importantly, exposes its symbolic state (path conditions plus symbolic stores) at arbitrary program points in a logical form that is directly reusable for specification synthesis. Crucially, QCP satisfies the symbolic-execution conditions of Proposition 1, thereby providing the foundation for the strongest-postcondition guarantee on loop-free segments. Other widely used alternatives such as Klee [?] are not built on Hoare logic and accordingly neither expose intermediate states at arbitrary locations nor present them in a logic-oriented form suitable for specification synthesis. QCP also has strong support for programs that manipulate complex data structures and ships with a library of verified inductive predicates and lemmas, which matches our goal of producing formally precise specifications. The gap QCP leaves — it requires user-supplied loop invariants and callee pre-/postconditions, and it may emit lemmas that need additional Coq interactive proofs — is exactly the gap filled by the other two components. The LLM proposes the missing invariants and specifications from the symbolic state QCP exposes; Frama-C/WP then discharges the resulting proof obligations automatically through SMT under the LLM-friendly ACSL surface syntax. The three components therefore split the work cleanly: QCP supplies the symbolic semantics, the LLM supplies the specification candidates, and Frama-C/WP closes the verification loop.

Stitching these three components together requires a translation layer, because QCP, the LLM, and Frama-C/WP each speak a different annotation language: QCPL (native to QCP), ACSL (native to Frama-C/WP), and natural-language prompts (for the LLM). The translator maintains a bidirectional bridge between ACSL and QCPL: it translates ACSL clauses into QCPL so that QCP can use them across loop and function boundaries, and it converts QCP's symbolic-state output into ACSL skeletons (paired with natural-language guidance for the LLM) and ACSL clauses (for Frama-C/WP verification). A deliberate design choice is that the LLM never sees QCPL; it only operates on ACSL plus natural-language guidance derived from the symbolic state. This avoids the common

failure mode of confusing two formally similar but semantically distinct languages, and the skeleton-plus-prose discipline ensures structural well-formedness before the LLM fills in the logical content. When verification fails, error information is translated back into natural language, together with its context, and routed into the next LLM call—closing the loop without exposing the verifier's internal representation to the model either.

B. Prompt Construction

Every LLM call in SESPEC follows a fixed template, but the concrete prompt is specialized to the input program. A lightweight static classifier inspects the source code and assigns each program to one of five routes: recursive data structure, recursive program, array, numeric, and plain, where plain is the default route for programs that do not need helper definitions. The selected route determines which worked examples, category-specific ACSL guidance, and route-specific rule fragments are inserted on top of the universal ACSL pitfalls block. Thus, the same template yields substantively different prompts for, say, a numeric recurrence and a linked-list cursor.

This routing drives the five LLM call sites in the pipeline: loop invariant generation, precondition generation, postcondition/assigns generation, syntax repair, and verifier-guided semantic refinement. In addition, the symbolic executor supplies the state-dependent content of each prompt — the variable state and the path constraints — together with an annotation skeleton to be completed by the LLM. At refinement time, the verifier's typed error categories are additionally folded into the prompt. All prompt templates are listed in the supplementary material.

VII. EXPERIMENTS

This section evaluates SESPEC, our symbolic execution-based approach to automatic specification generation. The main experiments use benchmark families from prior invariant and specification generation work, which enables direct C/ACSL comparisons under established settings. Section VIII

complements this evaluation with a separate goal-independent study, where assertions are removed and the generator must decide what functional behavior to specify.

A. Experimental Setup

Benchmark: We evaluate our approach on benchmark families commonly used by recent specification and invariant generation tools. These include: **SyGuS** [?], collected by CODE2INV [?]; **OOPSLA** [?]; **NLA**, collected by LIPuS [?]; the datasets from **SV-COMP** and **Frama-C**, collected by AUTOSPEC [?]. In addition, we constructed three new benchmarks to capture more program scenarios: The **pIp** set consists of small C fragments adapted from a navigation component of an industrial control codebase, involving pointer and struct manipulations as well as branching structures. We use **pIp** as a small, self-contained industrial sample rather than as a claim about the complexity or scale of the original system. We carefully adapt and reformulate **pIp** programs into a format acceptable to Frama-C and QCP, thereby establishing a new dataset, which constitutes one of our contributions; the **list-S** set is derived from the LIG-MM [?] benchmark and includes all singly linked list programs from SLING and SV-COMP; and the **numer-S** set is a curated collection of additional programs, comprising 20 linear and 20 nonlinear loops, primarily drawn from CLAUSE2INV [?] and harder than those in **SyGuS** and **NLA**.

Verifier: All experiments are performed using Frama-C version 27.1 (Cobalt). We employ the WP plugin as the verification backend, which enforces a 10-second timeout per query, uses Z3 as the primary SMT solver, and applies the Typed memory model.

Model: Our LLM selection for evaluating specification generation balanced model maturity, prevalence, and cutting-edge performance: **GPT-3.5-turbo** is an earlier, foundational model that serves as an essential baseline; **GPT-4o-mini**, a smaller, established model optimized for efficiency; **GPT-4o**, a widely adopted model for general code comprehension and generation and **Claude 3.7 Sonnet**, a stronger hybrid model with advanced reasoning and code understanding. For all experiments, we use a fixed sampling temperature of 0.7 to balance output diversity and stability.

Below we evaluate our approach through four research questions:

- **RQ1:** Which program classes can SESPEC handle?
- **RQ2:** What syntactic correctness, validity, and accuracy does SESPEC achieve?
- **RQ3:** How much do symbolic execution and refinement each contribute?
- **RQ4:** Does SESPEC outperform LLM- and SMT-based baselines?

B. RQ1: Which program classes can SESPEC handle?

Table II summarizes the 395 benchmark programs in our evaluation. They span linear, non-linear, nested/multi-loops, function calls, pointer manipulations, and recursive data types such as linked lists. The suite includes both loop-only programs (blue cells, invariant generation only) and programs

requiring full specifications (red cells, loop invariants plus pre/postconditions).

SESPEC verified 353 of 395 programs (89.4%) across numerical loops, pointer/struct fragments, linked lists, and **pIp** fragments adapted from an industrial navigation control codebase. The core symbolic execution algorithm does not systematically handle arbitrary recursion or external library/system calls; the LLM fallback covers some simple recursive cases (see Section IX), and broader support is left to future work. These failures reflect programs that are difficult for both the LLM and the SMT backend. For nested and multi-loop programs, symbolic execution often cannot obtain precise loop preconditions, so the LLM must infer relations across loop phases or loops. For non-linear programs, the missing facts often require non-linear arithmetic reasoning, and in some cases helper functions or inductive definitions that SMT solvers cannot prove within the budget. Recursive data structures similarly require inductive predicates beyond the finite flattened-memory summary. External and recursive calls fail for a different reason: they fall outside the symbolic-execution fragment, leaving no exact symbolic anchor for synthesis.

Lines of code do not reliably reflect difficulty on these benchmarks — the binding constraint is invariant complexity. SESPEC reaches 96% on Linear and Iteration-Free programs but drops to 77% on Non-Linear loops and 63% on Nested/Multi-Loops, where invariants must capture richer numerical relations, data-structure properties, and inter-loop interactions. The 0% on External & Recursive Call (3 programs) is the largest residual gap and is consistent with the scope discussion in Section X.

Finding (RQ1). SESPEC produces a *Valid* specification for 89.4% (353/395) of programs spanning linear/non-linear loops, memory, and linked lists; the residual failures concentrate on reasoning-heavy cases involving nested/multi-loop relations, non-linear arithmetic, recursive data structures, or external/recursive calls outside the symbolic-execution fragment.

C. RQ2: What syntactic correctness, validity, and accuracy does SESPEC achieve?

We evaluate correctness through three metrics:

- **Syntax rate (Syn.):** the percentage of generated specifications that are syntactically accepted by the ACSL/Frama-C front end;
- **Validity rate (Val.):** the percentage of generated specifications that are syntactically correct and whose generated clauses are accepted by Frama-C/WP under the verifier configuration in Section VII. For benchmarks that contain pre-existing assertions or verification targets, those targets are excluded from Val.; Val. measures whether the generated specification itself can be verified. For example, generated loop invariants are valid only if they satisfy both the *Base* and *Preservation* conditions;
- **Accuracy rate (Acc.):** the percentage of generated specifications that are syntactically correct, valid, and strong enough to prove the benchmark verification goal. For pre/postconditions, this means that they imply the given

TABLE II
DATASET OVERVIEW BY PROGRAM TYPE.

Program Type	SyGuS	OOPSLA	SV-COMP	NLA	numer-S	Frama-C	pIp	list-S	#Program	#Solved
Linear Loop	133	30	14	0	20	7	0	0	204	196
Non Linear Loop	0	0	0	30	20	3	0	0	53	41
Nested & Multi Loop	0	16	7	0	0	1	0	0	24	15
Memory & Loop	0	0	2	0	0	17	12	24	53	45
Iteration Free	0	0	0	0	0	20	38	0	58	56
External & Recursive Call	0	0	0	0	0	3	0	0	3	0
#Program	133	46	21	30	40	51	50	24	395	
#Solved	132	40	16	25	34	39	44	23		353
Avg. LoC	19	24	22	21	19	18	57	24		

TABLE III
SPECIFICATION GENERATION PERFORMANCE UNDER PASS@1, PASS@3, AND PASS@5.

Benchmark	Model	Pass@1 (%)			Pass@3 (%)			Pass@5 (%)		
		Syn.	Val.	Acc.	Syn.	Val.	Acc.	Syn.	Val.	Acc.
SyGuS	GPT-4o-mini	90.98	87.97	78.20	97.74	95.49	88.72	98.50	96.99	90.23
	GPT-3.5-turbo	93.23	88.72	84.21	97.74	97.74	92.48	98.50	98.50	93.23
	GPT-4o	98.50	98.50	93.98	99.25	99.25	99.25	99.25	99.25	99.25
	Claude 3.7 Sonnet	100.00	100.00	99.25	100.00	100.00	99.25	100.00	100.00	99.25
OOPSLA	GPT-4o-mini	80.43	65.22	32.61	89.13	86.96	43.48	89.13	86.96	47.83
	GPT-3.5-turbo	82.61	54.35	41.30	86.96	65.22	45.65	86.96	67.39	47.83
	GPT-4o	91.30	69.57	60.87	95.65	76.09	73.91	100.00	82.61	76.09
	Claude 3.7 Sonnet	93.48	69.67	63.04	97.83	80.43	76.09	97.83	93.57	86.96
SV-COMP	GPT-4o-mini	61.90	61.90	28.57	80.95	80.95	42.86	85.71	80.95	57.14
	GPT-3.5-turbo	95.24	90.48	66.67	95.24	90.48	66.67	95.24	90.48	66.67
	GPT-4o	90.48	90.48	52.38	100.00	100.00	71.43	100.00	100.00	76.19
	Claude 3.7 Sonnet	100.00	100.00	71.43	100.00	100.00	71.43	100.00	100.00	76.19
NLA	GPT-4o-mini	90.00	83.33	76.67	90.00	83.33	76.67	90.00	83.33	76.67
	GPT-3.5-turbo	76.67	70.00	50.00	80.00	80.00	56.67	80.00	80.00	56.67
	GPT-4o	93.33	90.00	83.33	96.67	96.67	86.67	96.67	96.67	86.67
	Claude 3.7 Sonnet	96.67	96.67	86.67	96.67	96.67	86.67	96.67	96.67	86.67
numer-S	GPT-4o-mini	95.00	90.00	72.50	100.00	95.00	75.00	100.00	95.00	75.00
	GPT-3.5-turbo	92.50	85.00	62.50	95.00	90.00	65.00	95.00	92.50	67.50
	GPT-4o	100.00	100.00	82.50	100.00	100.00	82.50	100.00	100.00	82.50
	Claude 3.7 Sonnet	100.00	97.50	85.00	100.00	100.00	85.00	100.00	100.00	85.00
Frama-C	GPT-4o-mini	54.90	49.02	47.06	62.75	50.98	49.02	64.71	50.98	49.02
	GPT-3.5-turbo	77.08	60.42	52.08	83.33	62.50	54.17	83.33	62.50	54.17
	GPT-4o	82.35	72.55	66.67	86.27	76.47	70.59	86.27	78.43	72.55
	Claude 3.7 Sonnet	86.27	78.43	72.55	92.16	82.35	76.47	92.16	84.31	76.47
pIp	GPT-4o-mini	90.00	87.50	65.00	92.50	92.50	72.50	92.50	92.50	75.00
	GPT-3.5-turbo	84.00	76.00	66.00	88.00	78.00	72.00	88.00	78.00	76.00
	GPT-4o	94.00	78.00	64.00	94.00	78.00	76.00	94.00	78.00	78.00
	Claude 3.7 Sonnet	100.00	88.00	82.00	100.00	90.00	88.00	100.00	92.00	88.00
list-S	GPT-4o-mini	62.50	41.67	33.33	75.00	45.83	41.67	79.17	54.17	45.83
	GPT-3.5-turbo	70.83	20.83	8.33	79.17	25.00	16.67	79.17	25.00	16.67
	GPT-4o	70.83	62.50	54.17	87.50	83.33	75.00	91.67	87.50	79.17
	Claude 3.7 Sonnet	95.83	83.33	75.00	100.00	100.00	95.83	100.00	100.00	95.83

target assertions (e.g., a functional property); for loop invariants, it further requires the *Termination* condition.

These metrics are assessed under Pass@k: a benchmark is counted as successful if any of up to k complete generation attempts satisfies the reported metric. Each attempt is a complete pipeline run and may include syntax repair, verification,

the internal refinement loop, and pruning; refinement iterations are not counted as additional Pass@k attempts. The Pass@1 cost measurements for the goal-independent study are reported separately in Section VIII-D.

Table III presents a comprehensive overview of our experimental results.

The observed gaps among different evaluation metrics reveal important characteristics of both benchmark difficulty and model capability. As benchmark complexity increases, the discrepancies between Syn., Val., and Acc. become more pronounced. A similar trend is observed across models: weaker models tend to exhibit larger gaps between these metrics. The starkest instance is GPT-3.5-turbo on **list-S**, where 70.83% syntactic correctness collapses to 20.83% validity and only 8.33% accuracy at Pass@1 — the model emits well-formed linked-list annotations that the verifier rejects — whereas Claude 3.7 Sonnet on the same benchmark reaches 95.83%/83.33%/75.00%. Notably, Val. does not trivially coincide with Syn. in our results, indicating that the generated specifications are not simplistic invariants (e.g., `true`) or trivial properties that programs would satisfy by default. Instead, our approach aims to capture non-trivial functional relations beyond syntax-valid or vacuous annotations, which naturally induces a meaningful distinction between syntactic correctness and semantic validity.

Importantly, our method demonstrates consistent effectiveness across different models and reduces the performance gap between stronger and weaker ones in several benchmark families. For example, on benchmarks such as **SV-COMP**, **NLA**, and **numer-S**, GPT-4o and Claude achieve identical performance at higher Pass@k levels (Pass@3 and Pass@5), suggesting that symbolic guidance can improve the usefulness of comparatively weaker models. Furthermore, the method exhibits rapid convergence under Pass@k evaluation. Performance improvements from Pass@3 to Pass@5 are marginal across several benchmarks, including **NLA**, **numer-S**, and **pIp**.

The results also indicate comparatively lower accuracy on the **OOPSLA**, **SV-COMP**, and **Frama-C** benchmarks. For **OOPSLA** and **SV-COMP**, this is primarily due to the presence of nested and multiple loops, where symbolic execution often fails to derive sufficiently strong preconditions. In addition, the absence of explicit loop conditions in **OOPSLA** further complicates invariant inference. In the case of **Frama-C**, the difficulty mainly stems from complex loop structures involving array manipulations, external function calls, and recursive behaviors, which remain challenging for symbolic execution. Despite these obstacles, our method still achieves notable performance on these challenging benchmarks. Moreover, the improvements from Pass@1 to Pass@5 show that additional attempts can help recover from difficult generation cases under the evaluated budget.

Finding (RQ2). Syntactic correctness is generally high, but the gap between validity and accuracy widens with benchmark complexity and narrows with model capability; SESPEC narrows model gaps on several benchmark families and reaches most Pass@k gains by Pass@3, with most residual accuracy loss on nested/multi-loop and memory-heavy families.

D. RQ3: How much do symbolic execution and refinement each contribute?

To assess the individual contributions of each component, we conducted an ablation study on 100 programs randomly

sampled from the benchmark families in Table II, following that table’s split into 70 loop-only programs (invariant generation only) and 30 programs requiring full specifications (function-level cases). For a fair comparison under controlled compute, all configurations are matched on LLM token usage and the refinement loop is capped at 3 iterations. This token-matched ablation isolates each component on the goal-dependent benchmarks; the complementary ablation in Section VIII re-measures the same components on the 400-program goal-independent benchmark without a compute cap.

We compare three configurations: **Only LLM (Pass@3)** uses the LLM alone for invariant generation without external guidance; **LLM + symbolic execution (Pass@3)** adds symbolic-execution-derived templates and program context; and **LLM + symbolic execution + refinement (Pass@1)** adds the verifier-guided refinement loop on top.

TABLE IV
ABLATION STUDY.

Method	Syn. (%)	Val. (%)	Acc. (%)
<i>Loop invariant generation (70 programs)</i>			
Only LLM	97.50	37.50	28.75
LLM + sym. exec.	95.71	62.86	60.00
LLM + sym. exec. + refine	95.71	95.71	87.21
<i>Program specification generation (30 programs)</i>			
Only LLM	66.67	26.67	16.67
LLM + sym. exec.	76.67	50.00	46.67
LLM + sym. exec. + refine	93.33	86.67	86.67

The ablation in Table IV shows that the LLM alone yields high syntactic correctness but low validity and accuracy. Adding symbolic execution lifts validity and accuracy substantially in both settings (average +28 percentage points, abbreviated pp) while reducing syntactic correctness only marginally on the loop-only setting (−1.79 pp). Adding the refinement loop pushes loop-only validity to 95.71% and lifts function-level validity from 50.00% to 86.67%, demonstrating that each component contributes a distinct and complementary gain. The largest individual jump comes from the refinement loop, which converts symbolic-execution-derived candidates that the verifier rejects into verifier-checked ones; this is consistent with the cost analysis in Section VIII, where refinement is also the dominant cost driver.

Finding (RQ3). Symbolic-execution guidance and verifier-guided refinement contribute distinct, complementary gains: over the LLM alone, symbolic execution adds ~28 pp of validity/accuracy, and refinement then lifts validity to 95.7% on loop invariants and to 86.7% on full specifications.

E. RQ4: Does SESPEC outperform LLM- and SMT-based baselines?

Table V is an effectiveness comparison under each method’s reported configuration, not a compute-matched comparison. SESPEC uses at most five complete generation attempts per program. The original AUTOSPEC study repeated each program five times, with iterative synthesis and verification within

TABLE V
ACCURACY COMPARISON ACROSS BENCHMARKS AND BASELINES. SESPEC REPORTS PASS@5 (CF. TABLE III); BASELINES USE THEIR BEST REPORTED CONFIGURATION.

Method	SyGuS	OOPSLA	SV-COMP	NLA	numer-S	Frama-C	pIp	list-S
CODE2INV	54.9%	—	—	—	—	—	—	—
HOLA	—	93.5%	—	—	—	—	—	—
CLN2INV	93.2%	—	—	—	—	—	—	—
LIPuS	93.2%	—	—	83.3%	—	—	—	—
CLAUSE2INV	99.2%	—	—	93.3%	—	—	—	—
AUTOSPEC-3.5	85.7%	82.6%	76.2%	70.0%	75.0%	60.8%	14.0%	0.0%
AUTOSPEC-4o	86.5%	71.7%	66.7%	70.0%	72.5%	68.6%	22.0%	0.0%
AUTOSPEC-Claude 3.7	88.7%	80.4%	66.7%	60.0%	55.0%	74.5%	4.0%	0.0%
ACInv-3.5	71.4%	45.7%	42.9%	—	—	34.5%	—	—
ACInv-4o	79.9%	76.1%	76.2%	—	—	48.3%	—	—
SESPEC-3.5	93.2%	52.2%	66.7%	56.7%	67.5%	51.0%	76.0%	16.7%
SESPEC-4o	99.2%	76.1%	76.2%	86.7%	82.5%	72.6%	78.0%	79.2%
SESPEC-Claude 3.7	100.0%	87.0%	76.2%	86.7%	85.0%	76.5%	88.0%	95.8%

each run [?]. ACInv also uses an internal augmentation loop; its published analysis reports settings of 1, 3, 5, and 10 augmentation cycles [?]. These counts do not equalize LLM calls, tokens, or verifier work across methods, and the numerical synthesis baselines do not have a directly comparable LLM-sampling budget.

Overall comparison with LLM-based approaches. Table V compares AUTOSPEC, ACInv, and SESPEC; SPECGEN is omitted here because it targets Java/JML rather than C/ACSL. On the four datasets originally introduced by AUTOSPEC, SESPEC achieves higher accuracy than ACInv on both GPT-3.5-turbo and GPT-4o, and both tools show similar gains from upgrading the model.

In contrast, AUTOSPEC barely improves from GPT-3.5-turbo to GPT-4o. Its reliance on few-shot prompting plus exhaustive sampling and verification-based filtering caps the gain that stronger models can deliver, so our AUTOSPEC-4o reproduction stays close to the reported AUTOSPEC-3.5 numbers.

SESPEC instead gains steadily with stronger models. Its workflow requires complex reasoning — program-execution simulation plus verifier-guided invariant refinement — which exceeds GPT-3.5-turbo’s instruction-following limits. Symbolic execution narrows this reasoning surface, so stronger models like GPT-4o benefit more.

The **pIp** fragments make this concrete: AUTOSPEC with Claude 3.7 Sonnet performs poorly despite Claude’s strong reasoning, because Claude struggles to emit syntactically correct ACSL, and a specification that does not parse is never verified. SESPEC avoids this with an ACSL skeleton built from the symbolic state, which both fixes the surface syntax and pins down the symbolic facts each clause must state; its advantage here is therefore semantic as well as syntactic.

Detailed results across benchmarks. Against SMT-based numerical baselines, SESPEC is on par with CLAUSE2INV on **SyGuS**, exceeds LIPuS on the nonlinear **NLA** benchmark while trailing CLAUSE2INV by 6.6 pp, and achieves 87% on the harder nested/multi-loop **OOPSLA** dataset. These SMT-based tools are specialized for numerical loops.

Against the LLM baselines AUTOSPEC and ACInv on numerical benchmarks, SESPEC scores 15% higher on **SyGuS** and higher on **NLA** and **numer-S**; on **OOPSLA** it matches ACInv and is only marginally below AUTOSPEC (with Claude 3.7 Sonnet, SESPEC is 5% higher). All three tools perform equally on **SV-COMP**, likely because the dataset is dominated by nested/multi-loop cases that no current approach solves. Overall, symbolic guidance captures specification facts that pure neural baselines miss on these benchmarks.

On **Frama-C**, SESPEC improves over AUTOSPEC via inductive-predicate support, which enables more expressive specifications. On the industrial **pIp** fragments (structs and pointer operations), the flattened memory model plus symbolic execution gives an average 50% higher success rate than AUTOSPEC. On the linked-list **list-S** benchmark, SESPEC also performs well; AUTOSPEC cannot handle this class because it lacks inductive-data-type support.

Finding (RQ4). SESPEC matches tools specialized for numerical invariants and clearly outperforms LLM-based generators (AUTOSPEC, ACInv), with the largest margins on the pointer-, struct-, and linked-list families that prior LLM tools handle poorly; its accuracy also scales with model capability where AUTOSPEC’s plateaus.

VIII. GOAL-INDEPENDENT EVALUATION

The previous section evaluates SESPEC under established benchmark protocols used by prior invariant and specification-generation tools. Many of those programs include explicit `assert` statements, which are useful for prior tools to generate specifications strong enough to discharge those assertions, i.e., verification goals, but they do not fully exercise the goal-independent setting SESPEC is designed to support. To complement that evaluation, we run a separate study on goal-independent programs, where all `assert` statements are removed and the generator must decide what functional behavior to specify. For this comparison we select the state-of-the-art specification-generation tool in each of the two language ecosystems we span: AUTOSPEC [?] for C/ACSL and SPECGEN [?] for Java/JML.

A. Experimental Setup

We construct **SESPEC-400**, a 400-program C benchmark built from three specification-generation suites: **FormalBench** [?] (250 programs) and **SpecGenBench** (59), translated to C from Java, and the native-C **SV-COMP** (91); programs are stripped of `assert` statements so the generator must decide what to specify. The **FormalBench** base dataset contains Java programs verified in OpenJML with sequential, branching, and loop-based structures. The translation preserves the source-level control flow and data dependencies used by the benchmark programs. Verification follows the Frama-C/WP protocol of Section VII (Z3, Typed memory model).

All LLM-based methods are evaluated on gpt-4o, gpt-5-mini, gpt-5.4-mini, and gpt-5 at sampling temperature 0.7 under Pass@1. We deliberately draw all four from a single model family: models from different families differ too much in training data and capability for a clean comparison, whereas staying within one family lets performance differences reflect reasoning capability rather than cross-family data bias. This set also differs from the models in Section VII, which were fixed by the baselines compared there.

We compare four C/ACSL workflows: full SESPEC (up to 10 refinement iterations), SESPEC *w/o refine*, SESPEC *w/o refine w/o symbolic execution (Pure LLM)*, and AUTOSPEC in its default one-shot configuration; plus SPECGEN, which is run on the original Java versions in its native Java/JML pipeline. The two ablations, *Pure LLM* and *w/o refine*, isolate the contributions of symbolic-execution guidance and verifier-guided refinement by comparison with the full pipeline, while AUTOSPEC serves as the verifier-level baseline that shares our C/ACSL setting. SPECGEN is the closest goal-independent system but runs in the Java/JML ecosystem, so we include it only as a cross-ecosystem reference rather than a direct verifier-level baseline. Each run is capped at a 1800-second outer timeout.

This study, run on all 400 **SESPEC-400** programs without the per-program compute cap of Section VII, probes the goal-independent setting along four orthogonal axes: whether the pipeline produces *valid* specifications, whether those specifications are *strong*, at what *cost*, and how it *fails*. Throughout, we say a program is *valid* when the generated specification is accepted in full by the verifier, i.e., *all* verification conditions it induces are discharged within the configured budget; a specification with any unproved obligation is not counted, so validity here measures whole-specification verification rather than partial proof. Extending the four research questions of Section VII, this study is organized around four further questions, one per axis:

- **RQ5:** How often does SESPEC produce a *Valid* specification?
- **RQ6:** How strong are SESPEC’s specifications versus the baselines?
- **RQ7:** What does SESPEC cost in time, tokens, and API calls?
- **RQ8:** What are SESPEC’s dominant failure modes?

TABLE VI
NATIVE-VERIFIER VALIDITY ON SESPEC-400.

Method	Model	Valid	Valid (%)
AUTOSPEC (C/ACSL)	gpt-4o	115	28.8
	gpt-5-mini	104	26.0
	gpt-5.4-mini	255	63.8
	gpt-5	288	72.0
SPECGEN (Java/JML)	gpt-4o	144	36.0
	gpt-5-mini	161	40.2
	gpt-5.4-mini	198	49.5
	gpt-5	251	62.7
SESPEC w/o refine w/o symbolic execution (Pure LLM)	gpt-4o	52	13.0
	gpt-5-mini	76	19.0
	gpt-5.4-mini	106	26.5
	gpt-5	55	13.8
SESPEC w/o refine	gpt-4o	126	31.5
	gpt-5-mini	107	26.8
	gpt-5.4-mini	132	33.0
	gpt-5	144	36.0
SESPEC	gpt-4o	272	68.0
	gpt-5-mini	333	83.3
	gpt-5.4-mini	373	93.3
	gpt-5	372	93.0

B. RQ5: How often does SESPEC produce a Valid specification?

Table VI reports the per-model results. Within the C/ACSL workflows, the full SESPEC pipeline achieves the highest *Valid* rate on every model: 68.0% on gpt-4o, 83.3% on gpt-5-mini, 93.3% on gpt-5.4-mini, and 93.0% on gpt-5. These correspond to absolute gains of +55.0, +64.3, +66.8, and +79.2 pp over Pure LLM, and +39.2, +57.3, +29.5, and +21.0 pp over AUTOSPEC. The ablations show two different sources of improvement. Refinement is the dominant source of validity gain, but it is also the most expensive component: enabling refinement lifts *Valid* by +36.5, +56.5, +60.3, and +57.0 pp across the four models, while requiring repeated LLM calls and verifier-feedback rounds. This confirms that iterative repair is highly effective, but its gain comes with substantial LLM-iteration cost.

By contrast, symbolic-execution guidance provides a more cost-effective source of improvement. Even without refinement, adding symbolic-execution information raises *Valid* over Pure LLM on every model: from 13.0% to 31.5% on gpt-4o, 19.0% to 26.8% on gpt-5-mini, 26.5% to 33.0% on gpt-5.4-mini, and 13.8% to 36.0% on gpt-5, whose unusually low 13.8% baseline is syntactic rather than semantic—it favors advanced ACSL constructs (behaviors, axiomatics) but often violates their surface syntax, triggering fatal whole-file parse errors that the symbolic-execution context helps avoid. Unlike refinement, this gain does not rely on repeated LLM sampling; it comes from exposing the local symbolic context before synthesis. Moreover, for loop-free programs fully supported by the flattened memory model, symbolic execution can directly compute the path-sensitive postcondition, so the core functional summary can be produced without an LLM call.

TABLE VII

PAIRWISE DIMENSION-WISE RELATIONS ON BOTH-VALID SETS, REPORTED AS LLM JUDGE → JUDGE + EXPERT REVIEW. DIRECTIONAL COLUMNS DENOTE THE MORE PERMISSIVE ACCEPTED DOMAIN FOR PRECONDITIONS AND THE LOGICALLY STRONGER PREDICATE FOR POSTCONDITIONS AND INVARIANTS. PRECONDITIONS AND POSTCONDITIONS COVER EVERY PAIR; INVARIANT COMPARISONS INCLUDE PAIRS FOR WHICH BOTH TOOLS PRODUCED LOOP INVARIANTS.

Baseline	Dimension	Evaluated	SESPEC side	Baseline side	Equal	Incomp.
AUTOSPEC (C/ACSL)	Preconditions	698	96→96	290→290	311→311	1→1
	Postconditions	698	342→342	18→18	320→320	18→18
	Loop invariants	540	120→120	50→52	5→5	365→363
SPECGEN (Java/JML)	Preconditions	673	161→164	178→174	221→222	113→113
	Postconditions	673	102→100	246→244	218→220	107→109
	Loop invariants	473	183→183	9→9	2→2	279→279

Fig. 3 isolates the effect of refinement budget on gpt-5.4-mini. The cumulative *Valid* rate rises from 24.2% at $k=0$ to 60.2% at $k=3$, 73.0% at $k=5$, and 79.2% at $k=10$, and then reaches 93.3% after the final Houdini pass. The marginal gain after $k=5$ is smaller, while Houdini contributes a final +14.1 pp by pruning failing clauses from the best refinement snapshots. In contrast, the *trivial refine* ablation, which uses only a plain error-plus-code prompt, reaches 70.0% under the same 10-iteration budget and final Houdini pass. This shows that differential refinement prompts and best rollback together account for roughly 23 pp of additional validity.

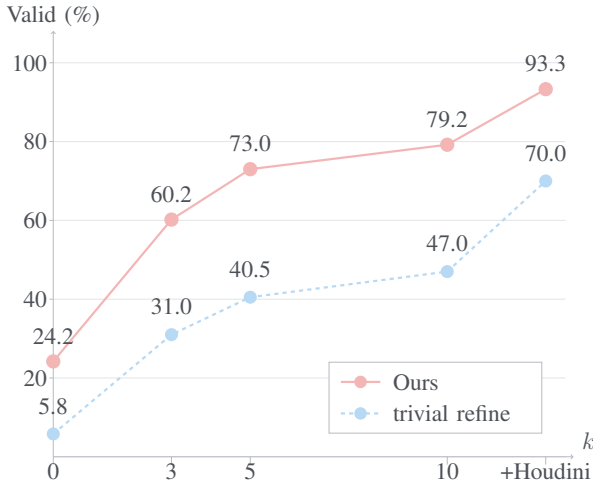


Fig. 3. Cumulative *Valid* rate on gpt-5.4-mini over 400 cases as the refinement budget k increases, for the full SESPEC pipeline (*Ours*) versus a *trivial refine* baseline.

Finding (RQ5). On SESPEC-400 the full pipeline reaches 68–93% *Valid*; refinement is the dominant but most expensive source of gain, whereas symbolic-execution guidance is a cheaper one that needs no repeated LLM sampling.

C. RQ6: How strong are SESPEC’s specifications versus the baselines?

Validity certifies that a specification is discharged by its verifier, but it does not rank how informative it is. We therefore compare accepted input domains (preconditions), normal-return guarantees (postconditions), and predicates at corresponding loop points (loop invariants) independently.

LLM judge and expert review. Each dimension uses a separate judge prompt and API call. Preconditions are classified by set inclusion between their accepted input domains; postconditions are compared by bidirectional implication over the common input domain; and invariants are compared at corresponding loop points. Thus, the two directional columns in Table VII mean more permissive for preconditions and logically stronger for postconditions and invariants. We do not aggregate them into an overall contract ranking. A missing precondition or postcondition denotes *true*. Expert review corrected 32 judge verdicts, shown as transitions in Table VII; any implication that cannot be established is included in *Incomp*. The supplementary material gives the prompts and reports the configuration, reviewer background, and A/B-order audit.

Table VII shows that SESPEC leads AUTOSPEC on postconditions and loop invariants. AUTOSPEC repeatedly samples the LLM and accumulates verifier-accepted *ensures* and invariant clauses, including many redundant, tautological, or trivially implied facts. SESPEC instead supplies the local symbolic state at each generation boundary, directing the model to generate path-sensitive relations and inductive predicates while reducing redundant clauses.

In the primary ordering, the baselines more often declare broader accepted input domains, frequently through a missing or *true* precondition, whereas SESPEC’s default preconditions explicitly record runtime- and memory-safety requirements. We report this as a difference in declared permissiveness rather than an intrinsic quality advantage: a broader declared domain is meaningful only when the additional inputs are valid executions for which the contract remains meaningful.

Against SPECGEN, SESPEC leads on loop invariants but SPECGEN leads on postconditions. The latter result is partly influenced by the Java/JML and OpenJML stack, whose managed arrays and object semantics support high-level functional contracts without the pointer-validity and buffer-related obligations required by ACSL and Frama-C/WP. In the C setting, these additional memory-model obligations can make some postconditions difficult for Frama-C/WP to discharge. As a result, SESPEC’s verification-driven pipeline may retain fewer postconditions when the corresponding proof obligations cannot be discharged within the allotted budget, whereas similar functional properties may be easier to verify in OpenJML. We

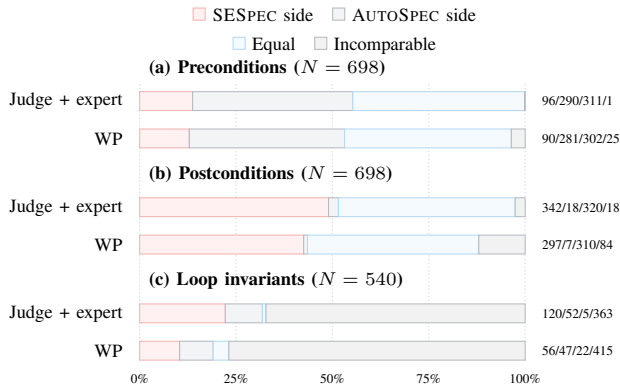


Fig. 4. Dimension-wise relations for AUTOSPEC versus SESPEC over identical data sets. A directional segment means a more permissive accepted domain for preconditions and a logically stronger predicate for postconditions or invariants. “Judge + expert” is the corrected judge result; WP checks the same implications. Right-hand counts follow the legend order.

therefore treat this cross-language result as contextual evidence rather than as a controlled comparison of the generation algorithms.

Frama-C/WP cross-check. We cross-check the same complete AUTOSPEC–SESPEC sets with separate Frama-C/WP harnesses: 698 pairs for preconditions and postconditions and 540 for invariants. With Frama-C 27.1, Alt-Ergo and Z3, and a five-second per-goal limit, the SESPEC/baseline/equal/incomparable counts are 90/281/302/25, 297/7/310/84, and 56/47/22/415, respectively. A direction fails unless WP proves every obligation; unsupported harnesses and unresolved loop correspondence are also included in Incomp. WP cannot distinguish a genuinely false implication from a true implication whose proof is incomplete; both situations are recorded as Incomp. Formal implication checking is constrained by ATP timeouts and automation limits, which leave substantially more pairs incomparable, especially for loop invariants. The LLM judge followed by expert review can inspect the semantic relations in these unresolved cases and therefore yields a more complete and fine-grained classification, while WP serves as an independent formal cross-check. Fig. 4 confirms the precondition and postcondition trends, while WP orders fewer invariants. We omit a WP cross-check for SPECGEN because translating Java/JML reference and array semantics into ACSL would confound the comparison.

Finding (RQ6). Symbolic context changes where specification strength appears: SESPEC yields more informative postconditions and stronger loop invariants than the compared systems, while the baselines’ broader declared input domains mainly reflect safety assumptions that are not written explicitly. Cross-language postcondition differences are partly attributable to Java/JML’s different semantic setting, so no single total ordering applies across all dimensions and languages.

D. RQ7: What does SESPEC cost in time, tokens, API calls, and memory?

This cost analysis is scoped to the **SESPEC-400** study in Section VIII; it is not collected over the established benchmark

families in Section VII. We use this scope because **SESPEC-400** is the goal-independent setting in which SESPEC, its ablations, AUTOSPEC, and SPECGEN are run under the same Pass@1 protocol. The paired comparison in Fig. 5 covers all 400 programs on gpt-5.4-mini and reports cost per attempted program, including failures. The original AUTOSPEC time, API-call count, and validity label are retained. Memory use is negligible: the process never exceeds ~ 500 MB peak RSS across the evaluated benchmarks.

Each solid segment in Fig. 5 sums the cost of valid runs and divides it by all 400 attempts; the hatched segment uses the same denominator for failed runs. Failures account for 519.4 of AUTOSPEC’s 932.1 seconds and 235.0 of SPECGEN’s 375.5 seconds per attempt, compared with 48.1 of SESPEC’s 318.2 seconds. Dividing total time by valid-run count gives 338.6 seconds per obtained valid SESPEC specification, versus 1,462.1 for AUTOSPEC and 758.6 for SPECGEN on this cohort. SESPEC’s 83.8k tokens per attempt include repeated refinement context; token counts apply no cache discount.

Finding (RQ7). SESPEC’s added framework complexity mainly appears as higher token use; its wall-clock time and API-call count are much closer to the baselines than the raw token totals alone suggest.

E. RQ8: What are SESPEC’s dominant failure modes?

To separate terminal outcomes from the errors seen in the detailed logs, Fig. 6 reports two views of an independent gpt-5.4-mini rerun on **SESPEC-400**, executed to capture per-case failure logs. This rerun yields 321 valid cases, fewer than the 373 (93.3%) of the main run in Table VI; the gap is intentional, as we cap this rerun at 3 refinement iterations (versus up to 10 in the main run) precisely to expose more failure cases for characterization. The left panel gives the final case-level outcome, while the right panel classifies the 79 non-valid cases (the 42 *No valid* plus 37 *No syntax* cases) by the dominant source program feature observed in the input and failure logs. Table VIII separately counts logged syntax events and prover failures by outcome (*Timeout*, *Unknown*, *Failed*) and by obligation content (*Loop Inv.*, *Post Cond.*, *Assigns*, and other verification conditions). The categories in Fig. 6 are mutually exclusive for counting: if several features occur in one program, we assign the case to the first feature that explains the blocking obligation in the log.

The event distribution in Table VIII shows where failed runs accumulate. Timeouts are dominated by loop invariant obligations (2,590/4,370 = 59.3%), especially preservation and establishment goals over arrays or heap-backed arrays. Postcondition and assigns timeouts are smaller but still substantial, indicating that the hard cases are not confined to one prompt or clause type.

The inspected logs contain *Timeout*, *Unknown*, and *Failed* outcomes, corresponding to proof timeouts, prover uncertainty, and backend or resource-level execution failures. These outcomes are not semantic counterexamples and cannot, by themselves, distinguish an incorrect generated specification from a correct but difficult proof obligation that the current

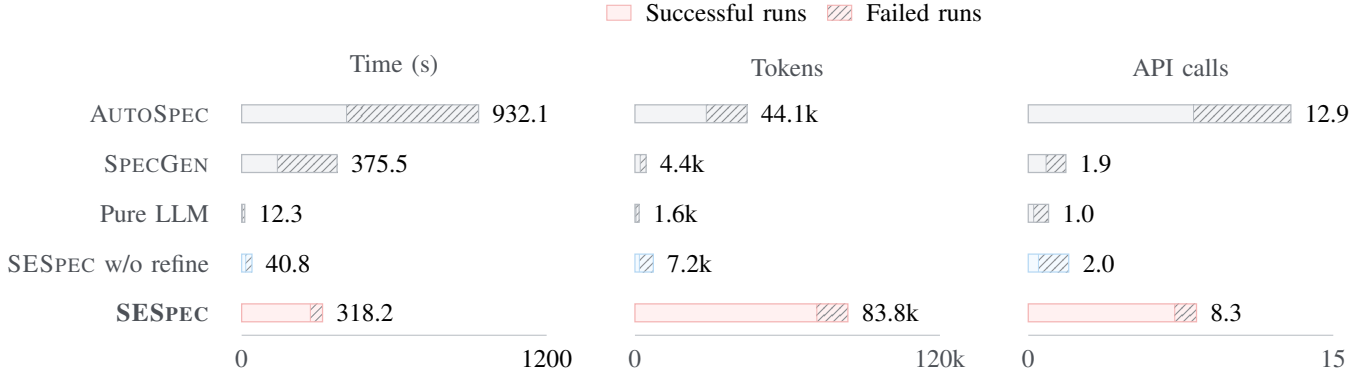


Fig. 5. Mean cost per attempted program on all 400 gpt-5.4-mini cases. Each bar stacks valid-run (solid) and failed-run (hatched) contributions, each divided by 400.



Fig. 6. Failure outcome and root-cause summary for the gpt-5.4-mini SESPEC-400 rerun.

TABLE VIII
VERIFIER LOG EVENTS FOR THE GPT-5.4-MINI SESPEC-400 RERUN.

Prover outcome	Loop Inv.	Post Cond.	Assigns	Other VC	Events
Syntax	—	—	—	—	2,315
Timeout	2,590	697	1,014	69	4,370
Unknown	62	4	111	0	177
Failed	27	22	0	0	49

verification backend does not discharge. In our manual review of the programs, generated specifications, and unresolved obligations, we more often observed candidate properties that were broadly consistent with the program but difficult for the current backend to prove, particularly for complex loop, array, and heap reasoning. This qualitative assessment does not rule out specification-synthesis errors in individual cases. The method and verifier boundaries exposed by these observations are discussed in Section X.

Finding (RQ8). Residual failures manifest predominantly as SMT timeouts on complex proof obligations. The logs cannot conclusively separate specification-synthesis errors from proof obligations beyond the current backend, although manual inspection more often points to verifier limitations.

IX. CASE STUDIES

The case studies below show, on concrete programs, what symbolic execution contributes to generated specifications. To keep the main text focused, we present five representative cases here and defer five supplementary cases to the supplementary material. Across all cases, SESPEC reuses the same

local symbolic context and template-routing signals, so each main case emphasizes a distinct role of that context rather than repeating the full mechanism.

Table IX summarizes the organization. The main cases cover phase-split loop invariants, path-complete postconditions under C aliasing, recursive heap shape specifications, the contrast between RTE-safety and functional specification, and the scale boundary on industrial control code. The supplementary material retains the factorial, GCD, MulLoop, Ackermann, and matrix-multiplication details as supplementary evidence.

TABLE IX
ORGANIZATION OF THE CASE STUDIES.

Case	Location	Main lesson
SyGuS loop	Main text	Phase-split invariant from weak goals
TripleAbsMax	Main text	Path-complete C/ACSL postconditions
Linked list	Main text	Shape-predicate routing
Linear search	Main text	RTE-safety vs. functional specification
Industrial pipeline	Main text	Scale and verifier boundary
Factorial, GCD	Supplementary	Inductive helper summaries
MulLoop	Supplementary	Prior-tool contextual example
Ackermann	Supplementary	ACSL/SMT recursion boundary
Matrix multiply	Supplementary	Nested loops; verifier/SMT-timeout boundary

In the figures below, ACSL annotations enclosed by $/ * @ \dots * /$ are generated by SESPEC and then retained after verification or pruning.

A. Case 1: Phase-Split Invariant from a Weak Goal

Fig. 7 isolates the benefit of symbolic execution when the assertion is too weak to guide invariant synthesis. The goal

Case 1: symbolic state exposes loop phases

```

int unknown();

/*@ requires n > 0; */
void SyGuS38(int n) {
    int c = 0;

    /*@
     loop invariant ((c == 0) && (n == \at(n, Pre))) || (1
     <= c <= n) ;
     loop invariant n == \at(n, Pre);
     loop assigns c ;
    */
    while (unknown()) {
        if (c == n) {
            c = 1;
        }
        else {
            c = c + 1;
        }
    }

    /*@ assert (c == n) ==> (c >= 0); */
}

```

Fig. 7. Case 1 (SyGuS-38): a nondeterministic loop whose phase structure is exposed by symbolic execution.

$(c = n) \Rightarrow (c \geq 0)$ can be proved by many uninformative invariants; what matters here is the loop’s phase structure.

At the loop boundary, symbolic execution separates the unentered state ($c == 0$) from the inductive region ($1 \leq c \leq n$) and records that n is stable. The disjunctive template instantiates exactly that split, yielding an invariant that describes the loop behavior rather than merely satisfying the final assertion.

B. Case 2: Path-Complete Postconditions for Pointer-Manipulating Code

Fig. 8, from our **pIp** benchmark, is the main C-specific case. The function computes the maximum absolute value of three struct-stored integers and writes it both to `pIp->tmax` and through `pIp->ret`.

Starting from the default precondition, including `\separated(pIp, pIp->ret)`, symbolic execution derives the guarded postconditions for the feasible paths.

C. Case 3: Shape-Predicate Routing for Linked Structures

Fig. 9 illustrates template routing rather than another scalar invariant pattern. The loop walks a singly linked list and returns the original head pointer, so ACSL needs an explicit heap-shape abstraction instead of only arithmetic facts.

Symbolic execution identifies `p` as the advancing list pointer and records the local update facts, while the static route recognizes the recursive type and field-access pattern. That route selects the recursive-shape template, which supplies `lseg`, `listrep`, and `list_length`; the loop template then connects the original head to the current suffix through `lseg(l, p)`. The key point is that the shape predicates are injected through a program-structure route and then anchored by symbolic cursor facts, rather than invented from source text alone.

Case 2: path-complete postconditions for pointer-manipulating code

```

typedef struct __TripleAbsMax
{
    int abs[3];
    int tmax;
    int* ret;
} TripleAbsMax;

/*@
 requires \valid(pIp);
 requires \valid(pIp->abs+(0..2)) ;
 requires \valid(pIp->ret);
 requires \separated(pIp, pIp->ret);

 ensures pIp->ret == \old(pIp->ret);
 ensures -pIp->abs[2] <= pIp->abs[1] && pIp->abs[0] <= pIp->abs[1] && pIp->abs
 [2] < 0 && pIp->abs[1] >= 0
 && pIp->abs[0] >= 0 ==> pIp->tmax == pIp->abs[1] && *\old(pIp->ret) == pIp->
 abs[1];
 ... all 32 paths
 assigns pIp->tmax, *(pIp->ret);
*/
void TripleAbsMaxFun(TripleAbsMax *pIp)
{
    int absfx1 = pIp->abs[0];
    int absfy2 = pIp->abs[1];
    int absfz3 = pIp->abs[2];

    if (pIp->abs[0] < 0) { absfx1 = -pIp->abs[0]; }
    if (pIp->abs[1] < 0) { absfy2 = -pIp->abs[1]; }
    if (pIp->abs[2] < 0) { absfz3 = -pIp->abs[2]; }

    if (absfx1 > absfy2) { pIp->tmax = absfx1; }
    else { pIp->tmax = absfy2; }

    if (absfz3 > pIp->tmax) { pIp->tmax = absfz3; }

    *(pIp->ret) = pIp->tmax;
}

/*@
 requires \valid(pIp);
 requires \valid(pIp->abs+(0..2));
 requires \valid(pIp->ret);
 requires \separated(pIp, pIp->ret);
*/
void main1(TripleAbsMax *pIp)
{
    pIp->abs[0] = 1;
    pIp->abs[1] = 2;
    pIp->abs[2] = -3;

    TripleAbsMaxFun(pIp);

    /*@ assert pIp->tmax == 3; */
}

```

Fig. 8. Case 2 (TripleAbsMax): a loop-free program with 32 feasible paths under the precondition shown.

D. Case 4: RTE Safety vs. Functional Specification

We reuse the linear search example from Fig. 3 of Preguss [?] and run SESPEC on the same source. The point is not a head-to-head accuracy comparison: Preguss is driven by RTE-safety obligations, while SESPEC attempts to generate goal-independent functional specifications. This distinction also affects failure handling. On the Frama-C validation side, SESPEC records potential RTE reports as warnings rather than treating RTE elimination as the main target. On the QCP side, however, an RTE can still block symbolic execution; the flattened-memory default precondition covers validity, bounds, and separation facts exposed by the model, but it does not guarantee that every arithmetic or pointer-related RTE is avoided. The refinement stage can ask the LLM to strengthen preconditions to avoid such failures, but this is a supporting mechanism for functional specification generation rather than the objective itself.

Fig. 10 shows the difference in target. Preguss emits the memory-safety precondition needed to discharge `arr[i]` accesses. SESPEC instead routes the loop to a count-occurrences helper and records both return cases: null means `val` is absent, and non-null means the returned pointer dereferences to `val` with no earlier occurrence. This illustrates why RTE-safety synthesis of Preguss [?] and our functional specification synthesis should not be read as identical tasks.

Case 3: route-selected shape predicates

```

struct SLL {
    struct SLL *tail;
    int head;
};

/*@
 predicate lseg{L}(struct SLL* x, struct SLL* y) =
   x == y || (x != y && \valid(x) && \separated(x, y)
   && lseg{L}(x->tail, y));
*/

/*@ predicate listrep(struct SLL* head) = lseg(head, \
null); */

/*@
 logic integer list_length{L}(struct SLL* x) =
   (x == \null) ? 0 : 1 + list_length{L}(x->tail);
*/

/*@
 requires listrep(l);
 requires \valid_read(&data);
 assigns \nothing;
 ensures listrep(l);
 ensures \result == l;
*/
struct SLL * main1(struct SLL *l, int data)
{
    struct SLL *p;
    p = l;

    /*@
     loop invariant listrep(p);
     loop invariant listrep(l);
     loop invariant lseg(l, p);
     loop invariant data == \at(data, Pre);
     loop invariant l == \at(l, Pre);
     loop invariant list_length(l) >= list_length(p);
     loop assigns p;
    */
    while (p) {
        if (p->head == data) {
            /*@ assert data == \at(data, Pre); */
            /*@ assert l == \at(l, Pre); */
            return l;
        }
        p = p->tail;
    }

    return l;
}

```

Fig. 9. Case 3 (linked-list traversal): a recursive data-structure program with shape predicates.

E. Case 5: Industrial Pipeline Stress Test

Program structure. We applied SESPEC to a 225-line C transcription of an industrial single-star-sensor Kalman-update pipeline. The translation unit contains 14 functions, 33 call sites, and 8 loops; the entry delegates to three pipeline stages, which in turn call ten numeric and quaternion helpers (Fig. 11). This case is included to show the scale boundary, not another single function invariant pattern.

What SESPEC produced. Driven by gpt-5.4-mini at maximum reasoning effort (reasoning_effort=high) in the goal-independent configuration, SESPEC completed specification generation for 13 of the 14 functions. The assembled file contains 198 ACSL clauses, and Frama-C/WP discharges 601 of 625 verification conditions (96.2%) at a 30s per-goal timeout. Table X shows that increasing the timeout from 5s to 30s recovers only one additional goal, so the residual failures reflect hard proof obligations rather than transient budget noise.

Because proof discharge alone does not measure

Preguss: RTE-guided safety specification

```

/*@ requires \valid_read(arr+(0..len-1));
 assigns \nothing; */
int * search(int * arr, int len, int val) {
    int i = 0;
    /*@ loop assigns i; */
    while (i < len) { ... }
}

```

SESPEC: verifier-checked prefix-search specification

```

/*@ logic integer count_eq{L}(int* a, integer lo,
 integer hi, integer v) =
   lo >= hi ? 0 : count_eq(a, lo, hi - 1, v)
   + (a[hi - 1] == v ? 1 : 0); */
/*@ requires \valid(arr + (0 .. len - 1));
 requires len >= 0;
 ensures \forallall integer k; 0 <= k < len ==>
   arr[k] == \at(arr[k], Pre);
 ensures \result == \null ==>
   count_eq(arr, 0, len, val) == 0;
 ensures \result != \null ==>
   count_eq(arr, 0, \result - arr, val) == 0
   && \result == val;
 assigns \nothing; */
int * search(int * arr, int len, int val) {
    int i = 0;
    /*@ loop invariant 0 <= i <= len;
     loop invariant \forallall integer k;
     0 <= k < len ==> arr[k] == \at(arr[k], Pre);
     loop invariant count_eq(arr, 0, i, val) == 0;
     loop assigns i; loop variant len - i; */
    while (i < len) {
        if (arr[i] == val) return arr + i;
        i = i + 1;
    }
    return (int *)0;
}

```

Fig. 10. Case 4 (linear search). Preguss verifies memory-access safety; SESPEC also captures the prefix-search property and returned-pointer consequence.

TABLE X
FRAMA-C/WP PROOF OBLIGATIONS FOR THE AEROSPACE PIPELINE.

Disposition	per-goal timeout		
	5s	10s	30s
Total	625	625	625
Valid	600	601	601
Unknown	2	2	3
Timeout	23	22	21
Discharge rate	96.0%	96.2%	96.2%

specification informativeness, we also manually audited the accepted contract of each function. We count a postcondition as functional only when it gives a non-vacuous relation between the output or return value and the inputs; ensures \true receives no such credit. Ten functions have functional postconditions: limit_abs, q_to_C, q_to_Ksi, triple_fabs_max, vec_cross3, vec_scale, vec_add, vec_sub, StsCompensate, and StsFilter. The other four lack a complete functional summary: StsModify provides only output bounds, mat_mul has the vacuous ensures \true, and the no-op helper q_normalize and the top-level CS_Sts_Gyro_Modify_Single_pipeline have no explicit accepted contract.

Boundary exposed. The functions with functional summaries are mostly arithmetic leaves whose semantics admit closed-form ACSL relations, such as limit_abs, vec_cross3, and q_to_C. The hard cases are delegate functions and matrix-style loops: composing multi-stage callee summaries and expressing row-major matrix products require either user-provided logic summaries or proof automation beyond the

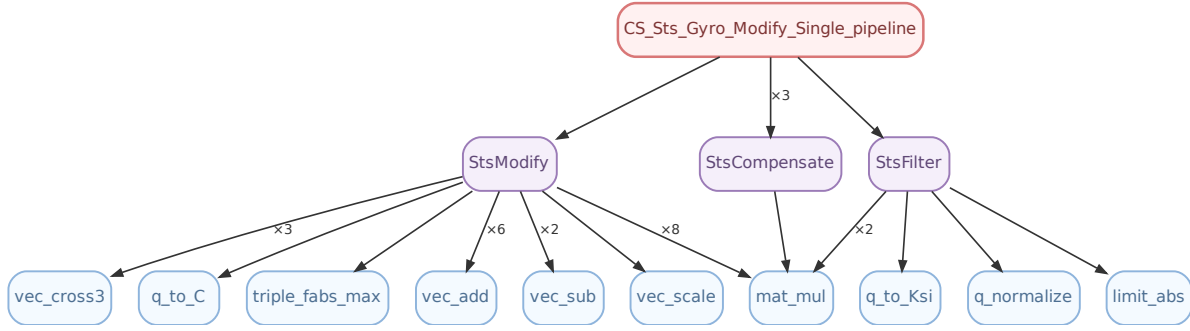


Fig. 11. Call graph of the single-star-sensor filter-update pipeline. Edge labels show call multiplicity; unlabeled edges are $\times 1$.

current SMT-only backend. Thus the 96.2% discharge rate measures verifier acceptance of the generated annotations, while the contract audit separately exposes how much functional behavior those annotations capture. Concretely, the undischarged goals concentrate in the matrix-multiply leaf `mat_mul` and its callers, exactly the triple-nested product dissected in the supplementary material, where the recursive `mat_sum` over nonlinear indices makes Z3 time out and the accepted specification degrades to `ensures \true`.

X. DISCUSSION: LIMITATIONS AND OPEN CHALLENGES

Scope and guarantees. SESPEC targets self-contained programs and does not model external or system calls, in particular dynamic memory allocation performed through library calls such as `malloc/free`. Programs that rely on such calls fall outside its scope: symbolic execution stops at such a call site and the affected segment is handled by LLM fall-back rather than summarized exactly. Furthermore, SESPEC’s strongest-postcondition guarantee, as stated in Section III and Proposition 1, holds only on covered loop-free segments under path-complete and precise symbolic execution in the flattened memory model. All other cases, including loops, recursive functions and data structures, are handled by verifier-checked best-effort specification generation, rather than exact symbolic summary computation. Crucially, verifier validity does not certify semantic completeness: a discharged specification may omit relevant functional behavior, especially when the required relations depend on unavailable symbolic states, unsupported heap abstractions, auxiliary lemmas, or reasoning beyond the configured backend. Extending the flattened memory model with allocation modeling and reusable library summaries is left to future work.

Scalability of symbolic execution. Symbolic execution remains difficult to scale to large programs: path explosion and the cost of carrying full symbolic stores both limit it in practice. Many symbolic executors mitigate this: separation logic and shape analysis let them reason about a class of heap-manipulating programs and recursive data structures such as lists and trees [?], while path abstraction and state merging curb path explosion [?], [?]. We deliberately choose

a basic symbolic executor instead, with the aim of obtaining precise symbolic states reusable for specification synthesis rather than scaling to larger programs. For what this basic executor cannot cover, we fully leverage the LLM: when the executor reaches a boundary, the framework asks the LLM to continue *in symbolic-execution order*, carrying the latest symbolic state forward and proposing a postcondition or loop invariant consistent with it. This yields stronger specifications than an unguided prompt, but it does not make LLM-generated summaries a sound replacement for symbolic execution beyond the supported fragment.

Verifier-backend ceiling. Two upstream limits cap what can be accepted even when the intended specification is semantically correct. First, some behaviors are cumbersome or unsupported in the ACSL fragment we use, such as lexicographic measures, higher-order quantified properties, and separation-logic-style abstractions over inductive heaps. Even a singly linked-list traversal may require user-defined `lseg/listrep` helpers that must be both faithful and SMT-friendly. Second, the ATP backend used by Frama-C/WP cannot reliably automate all quantified, inductive, and memory-heavy obligations generated by these specifications. *Timeout* events observed in our experiments are dominated by loop-invariant obligations, especially preservation and establishment goals over arrays or heap-backed arrays. WP often translates these obligations into quantified formulas over typed-memory stores, address shifts, integer casts, and occasionally recursive logic helpers such as counting functions, so even plausible invariants can exceed the configured per-query solver budget. *Unknown* and *Failed* events observed in our experiments mostly occur in verifier/modeling regions where the backend returns no definitive counterexample: whether the remaining obligations come from specification errors or verifier/modeling limitations remains unresolved.

Together with the program-feature breakdown in Fig. 6, this identifies *the SMT solving capacity of the ATP backend* as the dominant limit on our validation pass rate. Migrating to an interactive theorem prover can make residual obligations dischargeable by allowing explicit proof scripts, and in our manual inspection the remaining obligations become reachable

once the LLM writes such scripts under human guidance. This does not eliminate the scalability problem: it shifts the bottleneck from ATP automation to proof construction and proof search. We therefore view ITP integration as a promising future direction for SESPEC, especially as LLMs become better at generating and searching proof scripts, but not as an already solved route to large-scale verification.

XI. THREATS TO VALIDITY

We discuss threats to validity along the standard construct, internal, and external dimensions.

Construct validity. Our metrics — syntactic correctness (*Syn.*), validity under Frama-C/WP (*Val.*), and goal accuracy (*Acc.*) — measure proof acceptance under a specific verifier configuration, not semantic adequacy in absolute terms. A verifier-checked specification is not necessarily as informative as a hand-written one, and an unchecked one is not necessarily semantically wrong: it may be unprovable under the ACSL fragment, memory model, or ATP backend discussed in Section X. Quality is also hard to compare when two valid specifications differ in strength, readability, framing precision, or coverage. To partially address this, the experiment in Section VIII adds an LLM-as-judge pairwise strength comparison on the both-valid intersection. This comparison is itself approximate: the judge is an LLM, may weight informativeness differently from a human expert, and offers no complete oracle for specification quality. Reported gains should therefore be read as evidence under the chosen metrics, rather than as a complete characterization of specification quality. Indeed, how to formally evaluate the quality of a specification beyond mere verifier acceptance remains an open problem, as recent efforts to benchmark and assess LLM-generated formal specifications make clear [?], [?].

Internal validity: LLM nondeterminism and Pass@k. LLM completions are stochastic. We fix the sampling temperature at 0.7 throughout, with OpenAI’s `reasoning_effort=low` on the reasoning-tier models (`gpt-5.4-mini`, `gpt-5`); the sole exception is the industrial case study in Section IX-E, which uses `reasoning_effort=high`. Because single-shot results vary across runs, we report Pass@1, Pass@3, and Pass@5 separately for the goal-dependent benchmarks (Table III); the goal-independent experiment in Section VIII reports Pass@1 only, as its per-attempt cost is substantially higher. The Pass@1→Pass@5 gap recovers some hard cases on **OOPSLA**, **Frama-C**, and **list-S**, but is already marginal by Pass@3 on **NLA**, **numer-S**, and **pIp**. Absolute numbers should therefore be read within the LLM’s natural variance under the studied configuration.

Internal validity: data leakage. Bias and data leakage from the LLM’s pretraining corpus are a concern, especially in program verification, where domain-specific data are scarce. Training-based methods and benchmark-specific retrieval pipelines are most vulnerable, since benchmark-adjacent corpora, curated examples, or synthetically generated programs may implicitly encode benchmark-specific patterns. SESPEC mitigates this by avoiding benchmark-specific training and dataset-dependent demonstrations: it does not fine-

tune the model or train auxiliary components on the evaluation benchmarks, and its fixed prompts draw runtime content from the target program’s symbolic facts and static features rather than from labels or expected specifications. This reduces but cannot eliminate the risk of bias from pretraining data or benchmark familiarity in general-purpose LLMs. The observed gains are therefore evidence for symbolic-execution-guided prompting under the studied settings, not a guarantee that no benchmark-related knowledge resides in the underlying models.

External validity: benchmark and baseline selection. Our benchmarks — the AUTOSPEC suite and external datasets from LIPuS and CLAUSE2INV — align with current specification- and invariant-generation research, but their small average size makes them representative of small and medium-size verification tasks rather than the full diversity of real-world C systems. The **SV-COMP** subset in Section VII is the AUTOSPEC-curated 21-program slice, not a uniform sample of the full SV-COMP suite. To stress-test SESPEC within this scope, we introduced **pIp**, an industrial-derived benchmark with pointer-, struct-, and branch-heavy code, and the larger 400-case **SESPEC-400**. We have not evaluated SESPEC on industrial codebases of hundreds of thousands of lines or on programs that depend on extensive external library calls; scaling to such settings is left as future work. This boundary is not specific to SESPEC: fully automatic verification of *functional* correctness still operates at the scale of small, self-contained programs, because the underlying problem is intrinsically hard. The first wall is the reach of automated theorem provers (ATPs), which timeout or return *Unknown* on the nonlinear, quantified, and recursively defined obligations that realistic code induces, as our own Frama-C/WP runs repeatedly show (Section X). Migrating these obligations to an interactive theorem prover (ITP) changes this boundary rather than removing it: an ITP offers a richer proof surface for obligations that ATP backends fail to discharge, but it also shifts the burden to proof construction and proof search. Recent LLM-driven ITP efforts therefore remain evaluated on relatively small programs, while also suggesting that this route may become more scalable as LLM proof-script generation and proof search improve [?], [?], [?], [?].

External validity: memory model and verifier soundness. The verifier-backend completeness limits are discussed in Section X; here we isolate the soundness assumptions behind the reported verification results. We use Frama-C/WP’s *Typed* memory model, which is known to be unsound for several C idioms, including pointer-to-pointer casts that violate effective types, accesses that reinterpret memory between different scalar types, reads or writes through unions that mix the active variant, and overlapping or misaligned accesses to the same region. The evaluated benchmarks use none of them: **SyGuS**, **OOPSLA**, **NLA**, **numer-S**, and **SV-COMP** are numerical loops without unions or type punning, while **Frama-C**, **pIp**, and **list-S** use structs, pointers, and singly linked lists but do not cast pointers across incompatible types or access overlapping union variants. Dynamic allocation through `malloc` is outside the modeling scope. Programs outside these memory-

model assumptions are outside the soundness envelope of this evaluation.

XII. RELATED WORK

Traditional approach. Traditional techniques for program specification and invariant inference can be broadly categorized into dynamic and static methods. Dynamic approaches infer specifications and invariants by analyzing runtime data collected during test execution. Daikon [?] executes test cases to gather program traces and produces a set of candidate invariants based on a predefined library of templates, which are subsequently filtered using the observed data. Sharma et al. [?] employ a similar approach to generate candidate invariants, but additionally verify their soundness using SMT solvers. In contrast, DIG [?] bypasses template-based filtering and instead directly infers invariants from execution traces by integrating techniques from equation solving, polyhedra construction, and theorem proving. Despite their practicality, dynamic methods are highly dependent on the coverage and quality of the test suites.

Static methods eliminate the dependency on test cases and explore a variety of techniques, including constraint solving [?], [?], abstract interpretation [?] [?], abductive inference [?], Craig interpolation [?], [?], [?], recurrence analysis [?], [?], [?], shape analysis, and other methods [?] [?]. Colón *et al.* [?] present the first complete invariant generation method based on Farkas' lemma and solves the non-linear invariant constraints via quantifier elimination, which is of double exponential time complexity. Some heuristics are proposed to promote the efficiency of solving the constraints [?]. Cousot *et al.* [?] use convex polyhedra along with transformations and widening operations to discover linear invariants based on abstract interpretation. However, due to the intrinsic nature of abstract interpretation, the efficiency of the analysis heavily depends on the choice of abstract domains, and moreover, the approach offers no theoretical guarantee on the accuracy of the generated invariants due to the precision loss in abstraction. Lin *et al.* [?] synthesize inductive invariants using Craig interpolants derived from reachability analyses, and nonlinear Craig interpolant generation was first studied in [?]. The recurrence-based methods [?], [?], [?] derive recurrence relations from loops and solve these recurrences to obtain closed-form solutions as invariants. A key limitation of this approach, however, is that it is restricted to the cases where the recurrences admit closed-form solutions or satisfy some strong conditions. Wu *et al.* [?] solve the strong and weak invariant synthesis problems of polynomial programs by combining invariant templates, semidefinite programming and constraint solving techniques. However, these traditional static methods are often not scalable and may require manual intervention. In contrast to these techniques, which are powerful yet often specialized for programs adhering to specific forms (e.g., certain types of loops or invariants), our approach leverages LLMs to cover a broader set of benchmark patterns in the evaluated C/ACSL setting. In particular, some of these works focus primarily on theoretical challenges, and as a result, often exhibit limited efficiency and are difficult to apply directly to invariant generation at the program scale.

Machine learning and SMT-based approach. Machine learning techniques and Large Language Models have been increasingly used for synthesizing loop invariants. Recent advancements [?], [?], [?], [?], [?], particularly in deep learning and reinforcement learning, have spurred new approaches for invariant and specification generation. CODE2INV [?], based on Counterexample-Guided Inductive Synthesis (CEGIS) [?], uses graph neural networks to learn neural representations of program structures. While CODE2INV employs a reinforcement learning approach, it often faces challenges with limited performance and sparse rewards. To address these issues, LIPuS [?] establishes a clearer division of labor between the capabilities of RL and SMT solvers and introduces a novel two-dimensional reward mechanism to alleviate the sparse reward problem of CODE2INV. CLN2INV [?] learns loop invariants with continuous logic networks. CLAUSE2INV [?] refines the guess-and-check paradigm into a generate-combine-check framework, employing an LLM to generate candidate clauses and a counterexample-driven combiner to synthesize them into loop invariants. Loopy [?] uses LLM-generated ACSL invariant candidates, Houdini-style filtering, and error-guided repair for C loop benchmarks checked by Frama-C. However, CLN2INV, LIPuS, and CLAUSE2INV are mostly confined to numeric programs and do not constitute end-to-end functional-specification generators; instead, they produce invariants in SMT format. Loopy moves to ACSL loop annotations, but it remains goal-driven loop-invariant inference rather than goal-independent functional specification generation. Moreover, these tools are goal-dependent, i.e., they rely on predefined verification goals to generate accurate specifications. Empirical numbers against CLN2INV, LIPuS, and CLAUSE2INV on SyGuS / NLA / OOPSLA appear in Table V (Section VII); SESPEC is on par with CLAUSE2INV on SyGuS and falls between LIPuS and CLAUSE2INV on the nonlinear NLA benchmark, while these numerical tools do not report numbers on the harder nested/multi-loop OOPSLA dataset that SESPEC covers.

LLM-based approach. Pei *et al.* [?] demonstrate that fine-tuning pre-trained LLMs on code can effectively predict program invariants, with better performance than Daikon when the set of available tests is limited. However, like Daikon, their approach does not provide formal guarantees for the generated invariants. SPECGEN [?] is a Java-oriented tool that employs a two-phase process to generate specifications: it first leverages an LLM to produce invariant candidates, and then applies mutation operators with a weighted heuristic strategy to generate verifiable specifications when initial generation fails. Preguss [?] adopts a divide-and-conquer strategy that leverages potential RTE assertions from abstract interpretation-based static analyzers to guide LLM-based synthesis of minimal specifications, particularly preconditions that represent the weakest constraints sufficient to guarantee RTE-freeness. AUTOSPEC [?] decomposes programs into smaller parts and uses carefully designed prompts including few-shot examples to guide an LLM in generating specifications, which are then verified using Frama-C. ACInv [?] uses static analysis to extract essential information about loops and data structures,

TABLE XI
POSITIONING OF SESPEC AGAINST REPRESENTATIVE SPECIFICATION AND INVARIANT GENERATION TOOLS.

Tool	Target	Goal	Method	Program Support	Array/Struct/Rec.	Validation
Daikon	invariants	test-driven	traces/templates	Java/C/C++ traces	support/no opt.	trace filter
LIPuS	loop inv.	goal-driven	SMT/RL clauses	C loops	unsupported	SMT
CLN2INV	loop inv.	goal-driven	continuous logic nets	C loops	unsupported	SMT
CLAUSE2INV	loop inv.	goal-driven	LLM clauses + CEX	C loops	unsupported	SMT
Loopy	loop inv.	goal-driven	LLM + Houdini repair	C loops	unsupported	Frama-C
AUTOSPEC	ACSL specs	goal-driven	decomposition + few-shot	C function/loops	support/no opt.	Frama-C
ACInv	ACSL specs	goal-driven	static abstractions	C function/loops	optimized	Frama-C
Preguss	ACSL specs	goal-driven	RTE alarms	C function/loops	support/no opt.	Frama-C
SPECGEN	JML specs	goal-independent	LLM + mutation	Java function/loops	support/no opt.	OpenJML
SESPEC	ACSL specs	goal-independent	symbolic states/templates	C function/loops	optimized	Frama-C

which is then embedded into prompts for an LLM to generate candidate invariants and abstract predicates. We provide a detailed comparison in Section VII with ACInv [?] and AUTOSPEC [?], both of which target C programs.

The closest conceptual neighbor to SESPEC is ACInv, because both approaches inject program-analysis information into prompts. The main difference is the source and precision of that information. ACInv uses static loop and data-structure abstractions, which are useful for describing local program shape but can lose path-dependent equalities, must-alias versus may-alias distinctions, and correlations across struct fields or pointer dereferences. SESPEC instead mines symbolic-state snapshots. These snapshots preserve the path conditions under which a relation holds, the concrete memory updates induced by each path, and separation constraints required by ACSL verification. This is why the template source in SESPEC is symbolic rather than fixed, dynamic, or purely static.

Preguss and SPECGEN are related but address different settings. Preguss synthesizes minimal specifications for runtime-error freedom from abstract-interpretation-generated RTE targets; this makes it goal-dependent and safety-oriented rather than a functional summarization method. In SESPEC, potential RTE reports on the Frama-C validation side are recorded as warnings rather than as the primary success criterion: the pipeline uses QCP symbolic execution to expose functional behavior, may stop and fall back when an RTE prevents symbolic execution, and asks the refinement LLM to strengthen preconditions when doing so helps avoid such failures.

Lean-based specification generation. A separate line of recent work pursues specification synthesis and end-to-end formal verification inside the Lean proof assistant. CLEVER [?] and VERINA [?] curate algorithm-level benchmarks for which the specification, the implementation, and the proof are all written in Lean, exploiting Lean’s dependent type theory and expressive tactic language to encode higher-order invariants, inductive predicates, and proof obligations that are awkward to express in ACSL or JML. These efforts demonstrate that, given enough expressive power on the logic side, LLMs can drive specification and proof synthesis for non-trivial algorithmic programs. The fundamental restriction, however, is that the implementation language is forced to be Lean itself: target programs must be written (or rewritten) in Lean’s functional, dependently-typed syntax, which rules out specifying and ver-

ifying legacy C, Java, or other production codebases without a full reimplementaion. SESPEC targets the opposite trade-off — it gives up some of Lean’s expressive richness in exchange for working directly over off-the-shelf C source, so that specifications can be attached to the implementation language in which industrial software is actually written.

Across all four lines, no prior system targets *goal-independent, verifier-checked functional* specifications for off-the-shelf C: invariant-centric tools including LIPuS, CLN2INV, CLAUSE2INV, and Loopy are goal-driven and largely numeric-loop oriented; Loopy moves to ACSL/Frama-C but still synthesizes loop invariants from verification goals; AUTOSPEC and ACInv still depend on a goal and inject only static or few-shot context; SPECGEN removes the goal but in the Java/JML ecosystem; and Lean-based tools require reimplementing the program in Lean. SESPEC occupies this gap by driving generation with path-sensitive symbolic state over native C source (Table XI).

XIII. CONCLUSION

We presented SESPEC, an end-to-end framework for generating verifier-checked C program specifications by grounding LLM synthesis in symbolic states and verifier feedback.

The experiments also expose the current boundary of this approach: as analyzed in Sections X and XI, residual failures manifest predominantly as timeouts on inductive, recursive, and heap-heavy proof obligations under the current Frama-C/WP backend. The logs do not fully distinguish synthesis errors from correct but difficult obligations, so further progress may require both improved synthesis and a wider verification surface.

Looking ahead, specification generation may become more agentic as reasoning models improve. Instead of following a fixed symbolic-execution-driven order, future systems could let an LLM agent decide when to invoke symbolic execution, strengthen specifications or introduce lemmas. The key principle should remain unchanged: every generated specification should be grounded in symbolic states and verifier feedback, rather than free-form generation.

DATA AVAILABILITY

All datasets, implementation code, experimental results, and supplementary materials related to this work are publicly

available at <https://github.com/taylor-swift-13/SESpec>.

APPENDIX A SUPPLEMENTARY CASE STUDIES

This appendix retains five compact cases that support Section IX without interrupting the main narrative. The factorial and GCD cases show successful inductive-helper synthesis; MulLoop shows sign-sensitive invariant synthesis; Ackermann marks a recursion pattern beyond the current ACSL/SMT automation envelope; and matrix multiplication shows a verifier-timeout boundary rather than a generation failure.

a) *Factorial: Guarded Inductive Helper.*: Fig. 12 complements the phase-split loop case in Section IX: the loop may be skipped when $n < 1$, and the entered region requires the inductive relation $f == \text{product}(1, i - 1)$. Symbolic execution provides the entered/unentered guard, while the static route selects the inductive-helper template that supplies the ACSL `product` definition used by both the invariant and the postcondition.

b) *GCD: Recursive Summary with a Simple Measure.*: Fig. 13 shows the recursion case where the current workflow succeeds cleanly. When symbolic execution reaches a recursive call, SESPEC switches to the recursion template, emits `gcd_logic` branch-by-branch, and uses `decreases a + b`. This case is useful mainly as a contrast to Ackermann in Appendix A-0d: GCD needs only a single-measure descent and a single-unfold style of reasoning.

c) *MulLoop: Signed Multiplication Loop.*: Fig. 14 computes $a \cdot b$ by addition or subtraction depending on the sign of b . The negative branch requires the asymmetric invariant $\text{res} == -i * a$; reusing the positive-branch form would make the functional postcondition fail. In our run, SESPEC retains `ensures \result == a * b` because the branch condition selects the correct invariant template.

Supplementary Case: guarded factorial helper

```
int unknown();

/*@
  logic integer product(integer begin, integer end) =
    (end < begin) ? 1 : product(begin, end - 1) * end;
*/

/*@
  requires \true;
  assigns \nothing;
  ensures (n < 1) ==> \result == 1;
  ensures (n >= 1) ==> \result == product(1, n);
*/
int factorial9(int n) {
  int i = 1;
  int f = 1;

  /*@
    loop invariant (1 <= \at(n,Pre)) ==> (1 <= i <= n + 1);
    loop invariant (1 <= \at(n,Pre)) ==> (f == product(1, i - 1));
    loop invariant (! (1 <= \at(n,Pre)) ==> ((f == 1) && (i == 1) && (n == \at(n,
      Pre))));
    loop invariant n == \at(n,Pre);
    loop assigns f, i;
  */
  while (i <= n) {
    f = f * i;
    i = i + 1;
  }

  return f;
}

void goo9() {
  int t = factorial9(5);
  /*@ assert t == 120;
*/
}
```

Fig. 12. Supplementary factorial case: a guarded loop invariant with a recursive product helper.

d) *Ackermann: Recursion Beyond ACSL/SMT Automation.*: Fig. 15 marks a boundary rather than a prompt failure. The generated helper follows the C branch structure, but Ackermann needs a lexicographic termination argument and nested induction that are not handled well by the current ACSL/SMT stack. A scalar decreases $m * 1000 + n$ is not a sound substitute for the true lexicographic descent once n grows, and the functional postcondition would require proof support beyond the single-unfold reasoning that succeeds for GCD.

Supplementary Case: recursive GCD summary

```
/*@
  logic integer gcd_logic(integer a, integer b) =
    (a == 0) ? b :
    (b == 0) ? a :
    (a == b) ? a :
    (a > b) ? gcd_logic(a - b, b) :
    gcd_logic(a, b - a);
*/

/*@
  requires \true;
  decreases a + b;
  assigns \nothing;
  ensures \result == gcd_logic(a, b);
*/
int gcd(int a, int b)
{
  if (a == 0)
    return b;

  if (b == 0)
    return a;

  if (a == b)
    return a;

  if (a > b)
    return gcd(a - b, b);
  return gcd(a, b - a);
}

int main2()
{
  int a = 98, b = 56;
  int c = gcd(a, b);
  /*@ assert c == 14;
  return 0;
*/
}
```

Fig. 13. Supplementary recursive GCD case: a recursive logic summary with a single arithmetic termination measure.

Supplementary Case: signed multiplication loop

```
int foo57(int a, int b);

/*@
  requires \true;
  assigns \nothing;
  ensures \result == a * b;
*/
int foo57(int a, int b) {
  int res = 0;
  if (b >= 0) {
    /*@
      loop invariant 0 <= i <= b;
      loop invariant res == i * a;
      loop assigns i, res;
      loop variant b - i;
    */
    for (int i = 0; i < b; i++) {
      res = res + a;
    }
  } else {
    /*@
      loop invariant 0 <= i <= -b;
      loop invariant res == -i * a;
      loop assigns i, res;
      loop variant -b - i;
    */
    for (int i = 0; i < -b; i++) {
      res = res - a;
    }
  }
  return res;
}
```

Fig. 14. Supplementary MulLoop case from SpecGenBench.

e) *Matrix Multiplication: Triple-Nested Loops Beyond Per-Loop Symbolic Context.*: Fig. 16 is the multi-loop boundary that Reviewer 2’s failure-case question targets. SESPEC emits the intended quantified postcondition and the matching outer-loop invariants, while the innermost `s == mat_sum(..., k)` invariant is discharged by one-step induction. The remaining quantified obligations time out in Z3 because they combine recursive logic unfolding, havoc-updated array memory, and non-linear index arithmetic; refinement therefore strips the unproved clauses and the accepted

specification collapses to verifier-valid but vacuous ensures `\true`. This case illustrates a proof-automation limit rather than a generation failure.

APPENDIX B FULL PROMPT TEMPLATES

This appendix lists the templates used by the main LLM call sites referenced in Section VI-B, abridged to their static skeleton and runtime slots (the verbose in-prompt admonitions are elided). Each template’s static body is shown in black; runtime-injected slots appear in gray with colored tags from the shared figure palette: **LOOP/SYMBOLIC** for loop-aware symbolic context, **ROUTE/CALL** for route and call-site context, and **VERIFIER/SPECIFICATION** for verifier errors and specification edits.

A. Loop-invariant generation

The example below is a filled loop-invariant prompt for a small branch-then-loop program, with the long example and guidance blocks elided.

Supplementary Case: Ackermann recursion boundary

```
int fool(int m, int n);

/*@
  logic integer ackermann(integer m, integer n) =
    (m == 0) ? n + 1 :
    (n == 0) ? ackermann(m - 1, 1) :
    ackermann(m - 1, ackermann(m, n - 1));
*/

/*@
  requires m >= 0 && n >= 0;
  decreases m * 1000 + n;
  assigns \nothing;
  ensures \result == ackermann(m, n);
*/
int fool(int m, int n) {
  if (m == 0) {
    return n + 1;
  }
  if (n == 0) {
    return fool(m - 1, 1);
  }
  return fool(m - 1, fool(m, n - 1));
}
```

Fig. 15. Supplementary Ackermann case. The candidate specification is syntactically clean but not discharged by Frama-C/WP under the current ACSL/SMT backend.

Supplementary Case: matrix-multiplication proof boundary

```
/*@
  logic integer mat_sum(int *A, int *B, integer ca_rb, integer cb,
    integer i, integer j, integer k) =
    k <= 0 ? 0 :
    mat_sum(A, B, ca_rb, cb, i, j, k - 1) +
    A[i * ca_rb + (k - 1)] * B[(k - 1) * cb + j];
*/

/*@
  requires \valid(out + (0 .. ra*cb - 1));
  requires \valid(A + (0 .. ra*ca_rb - 1));
  requires \valid(B + (0 .. ca_rb*cb - 1));
  requires \separated(out + (0..ra*cb-1), A + (0..ra*ca_rb-1));
  requires \separated(out + (0..ra*cb-1), B + (0..ca_rb*cb-1));
  // default precondition also bounds every cell to [0,100] and each
  // dimension product to (0,100), keeping the arithmetic overflow-free
  assigns out[0 .. ra*cb - 1];
  ensures \forall integer i, j; 0 <= i < ra && 0 <= j < cb ==>
    out[i*cb + j] == mat_sum(A, B, ca_rb, cb, i, j, ca_rb); // Z3 TIMEOUT (10 s)
*/
void mat_mul(int *out, int *A, int *B, int ra, int ca_rb, int cb) {
  /*@
    loop invariant 0 <= i <= ra;
    loop invariant \forall integer ii, jj; 0 <= ii < i && 0 <= jj < cb ==>
      out[ii*cb + jj] == mat_sum(A, B, ca_rb, cb, ii, jj, ca_rb); // Z3
      TIMEOUT (10s)
    loop invariant \forall integer p; i*cb <= p < ra*cb ==> out[p] == \at(out[p], Pre); // Z3 TIMEOUT
    loop assigns i, out[0 .. ra*cb - 1];
  */
  for (int i = 0; i < ra; i++) {
    /*@
      loop invariant 0 <= j <= cb;
      loop invariant \forall integer jj; 0 <= jj < j ==>
        out[i*cb + jj] == mat_sum(A, B, ca_rb, cb, i, jj, ca_rb); // Z3
        TIMEOUT (10s)
      loop assigns j, out[i*cb .. i*cb + cb - 1];
    */
    for (int j = 0; j < cb; j++) {
      int s = 0;
      /*@
        loop invariant 0 <= k <= ca_rb;
        loop invariant s == mat_sum(A, B, ca_rb, cb, i, j, k); // DISCHARGED (
        one-step induction)
        loop assigns k, s;
      */
      for (int k = 0; k < ca_rb; k++)
        s = s + A[i*ca_rb + k] * B[k*cb + j];
      out[i*cb + j] = s;
    }
  }
}
```

Fig. 16. Supplementary matrix-multiplication case ($ra \times ca_rb$ by $ca_rb \times cb$ product). SESPEC generates the correct quantified specification, but Frama-C/WP→Z3 times out on the outer invariants and postcondition, so the accepted specification degrades to ensures `\true`.

Loop-invariant generation prompt

```
### Role / Task ###
Given a C program with a loop, generate ACSL loop invariants
that let Frama-C verify it.

ROUTE examples + ACSL helper policy for the matched route, omitted
GUIDANCE strength / supplementary / assertion guides, omitted

### Rules ###
- Only fill the loop-invariant PLACE_HOLDERS (or add a helper block
  before the function); keep existing annotations unchanged.
- Use the pre-condition, not \at(var, LoopEntry).
- Emit one self-contained ``c`` block.

SYMBOLIC STATE supplied by the symbolic executor
Pre-condition:
  (\at(a,Pre)==0 && y==0 && x==1 && a==\at(a,Pre))
  || (\at(a,Pre)!=0 && y==0 && x==0 && a==\at(a,Pre))

Loop + skeleton to complete:
/*@
  loop invariant (\at(a,Pre)!=0) ==> ((y<100000) ==>
    ↪ PLACE HOLDER x);
  loop invariant (\at(a,Pre)!=0) ==>
    (! (y<100000) ==> (y==0 && x==0 && a==\at(a,Pre)));
  loop invariant (\at(a,Pre)!=0) ==> (a==\at(a,Pre));
  ... (symmetric a==0 branch omitted) ...
*/
while (y < 100000) { x = x + y; y = y + 1; }
```

B. Precondition fallback

Precondition fallback prompt

```

### Role: ###
Generate ONLY function-level preconditions plus supporting
predicate/logic declarations needed by them.

### Goal: ###
Rule out Frama-C runtime errors:
- pointer dereferences must be covered by \valid or
  \valid_read;
- array ranges must cover all indexed accesses;
- inductive data structures use structural predicates from
  examples.

ROUTE EXAMPLES
{examples}

### Output format: ###
Output exactly one fenced ``acsl`` block:
/*@ predicate <name>(...) = ...; */
/*@ logic <type> <name>(...) = ...; */
/*@
  requires <clause>;
*/

### Hard rules: ###
- Output only ACSL annotations.
- Produce requires only; no ensures, assigns, or loop invariant.
- Each predicate or logic declaration lives in its own /*@ ... */
  ↪ block.

Function under specification:
{code}

```

C. Postcondition and assignment generation

Postcondition / assignment generation prompt

```

### Task: ###
Transform the following C code into an ACSL-annotated version.

### Rules: ###
- Only fill ensures PLACEHOLDER and assigns PLACEHOLDER.
- The input is a complete C file; callee specifications are
  read-only context.
- Output only the modified target function block. The
  harness splices it back into the original file.
- Helper predicate/logic definitions are allowed and required
  if the target specification references a name not already defined.
- Replace the ensures PLACEHOLDER with as many strong,
  ↪ non-redundant
  clauses as the behavior admits (typically 3-8): the closed-form
  result, per-branch facts, and frame consequences.
- Inline relations directly in ensures; add a predicate/logic helper
  only when the relation is recursive or higher-order.

LOOP-AWARE DERIVATION
{loop_guide}

### ACSL pitfalls ###
UNIVERSAL + SYMBOLIC-ROUTE HINTS
{category_hints}

RESOLVED ARRAY SIZES
{array_length_facts}

### C Program Code: ###
{code}

```

D. Syntax repair

A deterministic ACSL fixer (Section V) runs before this prompt and clears the common mechanical faults; the template below is the LLM fallback used only when the fixer cannot resolve the parser error.

Syntax-repair prompt

```

### Role: ###
You are an expert in C and Frama-C.

### Task: ###
Correct syntactically incorrect ACSL annotations using
Frama-C errors.

Syntax error:
VERIFIER ERROR
{error_str}

C code with incorrect ACSL annotations:
{c_code}

### ACSL pitfalls ###
UNIVERSAL + SYMBOLIC-ROUTE HINTS
{category_hints}

### Rules: ###
- Do not modify function bodies or signatures.
- Only change ACSL annotations.
- New top-level predicate/logic blocks may be added before
  the function.
- If an unbound logic/predicate name is reported, define it
  or remove every clause that references it.

```

E. Pairwise specification strength judges

Fig. 17 gives the three independent judge prompts used for Table VII. Each invocation receives only the target function's annotation blocks relevant to that dimension; the complete prompt text and structured responses are included in the artifact.

Precondition judge

```

[System]
Compare only accepted input domains. A missing
precondition means true. Report which predicate is
WEAKER, i.e., which one accepts more inputs.
Do not compare postconditions or invariants.

[User]
Evaluate independently:
  B_to_A: P_B implies P_A
  A_to_B: P_A implies P_B
B_to_A only => A_accepts_more
A_to_B only => B_accepts_more
both => equivalent
neither/unknown => incomparable
Return both directions, the relation, and a concise
implication-based reason as strict JSON.

```

Postcondition judge

```

[System]
Compare only normal-return guarantees on the common
accepted input domain. Missing preconditions or
postconditions mean true. Ignore invariant content.

[User]
Let D = P_A AND P_B. Evaluate independently:
  A_to_B: D AND Q_A implies Q_B
  B_to_A: D AND Q_B implies Q_A
A_to_B only => A_guarantees_more
B_to_A only => B_guarantees_more
both => equivalent
neither/unknown => incomparable
Do not reward a wider input domain or infer an
unstated guarantee from the body. Return both
implication directions, relation, and reason as JSON.

```

Loop-invariant judge

```

[System]
Compare only explicit loop invariants at corresponding
program points. Ignore function contracts.

[User]
At each point, conjoin that side's invariants.
Evaluate independently at every point:
  A_to_B: I_A implies I_B
  B_to_A: I_B implies I_A
A_to_B only => A_invariants_stronger
B_to_A only => B_invariants_stronger
both everywhere => equivalent
neither/unknown => incomparable
Do not merge different loop points or infer reachable
states from bodies. Return both implication directions,
relation, and a concise decisive reason as JSON.

```

Fig. 17. Independent strength-judge prompts used before expert review. A and B denote two anonymized specifications; tool identities are not disclosed to the judge.

F. Verifier-guided semantic refinement

Fig. 18 gives the refinement prompt used by the verifier-guided refinement loop of Section V.

```

Verifier-guided refinement prompt

### Task: ###
Frama-C reports verification failures grouped by category. Each
↪ category
has a different fix strategy; only relevant strategies are shown.

### Categorized errors ###
ACTIVE ERROR BUCKETS
{error_str}

### Per-category fix strategy ###
ACTIVE STRATEGIES ONLY
{strategy_blocks}

### Current C code with annotations ###
{c_code}

LOOP-AWARE DERIVATION
{loop_guide}

### ACSL pitfalls ###
UNIVERSAL + SYMBOLIC-ROUTE HINTS
{category_hints}

### Output rules ###
- The target function's requires may be strengthened when necessary.
EDIT MODE
{output_format_block}
- Do not modify user assert statements.
- Do not modify correct loop invariants or correct postconditions.
- Wrap the corrected target block, or the complete file in
↪ full-file mode,
in one ```c``` fenced block.

```

Fig. 18. Verifier-guided semantic refinement prompt.

APPENDIX C

JUDGE CONFIGURATION AND POSITION-BIAS AUDIT

We used GPT-5.4 with a 2,048-token output limit, no input truncation, and the provider-default temperature, top- p , and reasoning effort. Each precondition, postcondition, and loop-invariant comparison used a separate API call and the corresponding prompt in Appendix B-E. The expert review was conducted by two master's-level researchers with formal-verification backgrounds.

The primary run placed the baseline in A and SESPEC in B. To test order sensitivity, we repeated every judge decision with SESPEC in A and the baseline in B, then mapped the output back to the original tool orientation. Table XII compares the raw judge results before expert correction. Pairs with no substantive predicate on either side are deterministically equal and are included in the reported agreement. The principal AutoSpec trends are stable. The SpecGen postcondition and invariant directions also persist, but its precondition direction reverses. Because Java/JML and C/ACSL have different expressive and semantic capabilities, a complete cross-language precondition comparison is not possible; this heterogeneity also amplifies positional bias in the judge.

TABLE XII
A/B-ORDER AUDIT OF THE LLM JUDGE. OUTCOME TUPLES ARE
SESPEC/BASELINE/EQUAL/INCOMPARABLE.

Dimension	Pairs	Agreement	Primary	Swapped
AUTOSPEC (C/ACSL)				
Preconditions	698 695	(99.6%)	96/290/311/1	99/288/310/1
Postconditions	698 672	(96.3%)	342/18/320/18	349/11/325/13
Loop invariants	540 435	(80.6%)	120/50/5/365	160/49/5/326
SPECGEN (Java/JML)				
Preconditions	673 546	(81.1%)	161/178/221/113	200/86/232/155
Postconditions	673 518	(77.0%)	102/246/218/107	141/203/217/112
Loop invariants	473 382	(80.8%)	183/9/2/279	198/3/1/271

APPENDIX D TRANSLATION DETAIL

We used GPT-5.4 to translate the 250 FormalBench and 59 SpecGenBench Java programs into C; the other 91 SV-COMP cases are native C. For each Java case, the model was asked to preserve the target method's branches, loops, assignments, and returns while adapting its types and memory operations to C. We removed JML annotations and goal assertions and renamed the target function to `foo` followed by its case ID. Programs that could not be translated into the supported C fragment and compiled were excluded; for example, SpecGenBench's `ConvertToTitle` uses `Java StringBuffer.append` and `reverse` and was not included. All 400 selected C programs pass GCC C11 compilation: 309 translated programs and 91 native-C programs.

In the resulting programs, Java `int`, `long`, and `boolean` are represented by C `int`, `long long`, and 0/1-valued `int`. An array parameter `int[] a` becomes `int *a`, `int a_len`; `a.length` becomes `a_len`, and array lengths are passed through calls. A local `new int[n]` becomes a C allocation with a stored length. Constants such as `Integer.MIN_VALUE` and the few string operations are adapted to their C counterparts.

Compilation checks syntax, not full semantic equivalence. Cross-language comparisons are restricted to normal executions with valid memory accesses, successful allocation, and no signed overflow.

We also checked translation behavior by differential execution on all 309 Java/C pairs. The check covered 4,325 comparable executions using deterministic edge-case and seeded pseudorandom inputs, and compared the return value and any modified integer-array arguments. The Java programs were compiled for Java 8, and the C programs were compiled with GCC C11 and undefined-behavior checks. Of these executions, 4,323 produced matching observations; the other two triggered C undefined behavior in the same program. Thus, 308 of 309 pairs showed no discrepancy on their tested inputs, while one pair exposed a cross-language semantic difference.

In that pair, case 112, input '0' makes the expression `1 << (c - 'a')` shift by a negative count in C, whereas Java defines the shift and returns 1. This discrepancy is outside the common defined-behavior domain, but it shows why compilation and finite-input agreement cannot establish unrestricted Java/C equivalence. The differential result is empirical evidence over the tested inputs, not a proof of semantic equivalence.